

A hands-on companion to the Lunar-HFE research code

Physics and Code for Beginners

Measuring how the Moon's soil conducts heat

Who this is for. You — someone who is smart and motivated but has *no prior background* in coding, numerical methods, or heat-transfer physics, and who wants to understand *this specific project* from the ground up.

The whole project in one sentence. We take the only two sets of underground temperature measurements ever made on the Moon (Apollo 15 and 17), build a small physics simulator of how heat moves through Moon soil, and tune one number — the deep thermal conductivity K_d — until the simulator matches the real measurements at each site.

Contents

How to read this guide	4
1 The big picture: what question does this repo answer, and why care?	5
1.1 The scientific question in one breath	5
1.2 Why this is hard, and why it matters	5
1.3 The headline result (the answer)	5
2 Notation reference — every symbol used in this project	6
2.1 Coordinates, time, and indexing	6
2.2 Heat, flux, and energy	7
2.3 Thermal conductivity model (Hayne 2017)	7
2.4 Surface radiation, albedo, and orbital forcing	8
2.5 Boundary conditions and problem setup	8
2.6 Skin depth, diffusivity, and diurnal damping	9
2.7 Numerical discretisation	9
2.8 Retrieval, data, and statistics	9
3 Physics fundamentals, from scratch	10
3.1 Heat vs. temperature (they are not the same thing)	10
3.2 Conduction: how heat moves through a solid	10
3.3 The 1-D heat equation — full derivation from scratch	11
3.3.1 The problem in one sentence	11
3.3.2 Picture a thin horizontal slice of soil	11
3.3.3 Heat flux at each face	11
3.3.4 Energy conservation (the bookkeeping step)	11
3.3.5 From two faces to a derivative	12
3.3.6 Reading the final equation, term by term	12
3.3.7 The two boundaries (summary)	12
3.3.8 What “solving” means in this repository	13
3.4 The three soil properties — what they mean and why the heat equation needs them	13
3.4.1 Thermal capacity: why ρc_p sits on the left	13
3.5 Thermal conductivity $K(T, z)$ — full physical picture	13
3.5.1 Structural part $K_c(z)$ — compaction with depth	14
3.5.2 Radiative boost — why T^3 ?	14
3.5.3 Density and specific heat	14
3.6 The thermal skin and skin depth — full derivation	14
3.6.1 Thermal diffusivity	14
3.6.2 The decaying thermal wave	15
3.6.3 Deriving the skin depth	15
3.7 Radiation, sunlight, and the geothermal gradient — full picture	15
3.7.1 Stefan–Boltzmann: why T^4 ?	15
3.7.2 Surface energy balance	16
3.7.3 Insolation model	16
3.7.4 Geothermal gradient — full derivation	16
3.8 Boundary conditions — full derivation and connection to the main equation . . .	17
3.8.1 Why the main equation is not enough	17
3.8.2 One unifying idea: energy balance at every control volume	17
3.8.3 Top boundary — radiative surface balance	17
3.8.4 Bottom boundary — geothermal flux	18
3.8.5 How the pieces fit in one timestep	18

3.9	Orbital mechanics and the top boundary — full picture with diagrams	18
3.9.1	The Moon in orbit: what “one day” means	18
3.9.2	From geometry to insolation: the cosine law	19
3.9.3	Top boundary: surface energy balance (control volume)	20
3.9.4	How orbital forcing connects to the heat equation	21
3.9.5	Bottom boundary (for completeness)	21
3.9.6	Code map: from <code>insolation</code> to <code>_step</code>	21
4	The numerical method, gently	21
4.1	Slicing the depth: why a <i>geometric</i> grid	22
4.2	Stepping through time: what a “timestep” is	23
4.3	Crank–Nicolson: a stable, accurate way to take a step	23
4.4	The tridiagonal (Thomas) solver: the fast inner engine	24
4.5	Newton’s method for the non-linear surface	24
5	The Anchor Point Method (the centerpiece)	25
5.1	The problem: the deep Moon settles agonizingly slowly	25
5.2	The key insight: two clocks	26
5.2.1	Where $d\langle T \rangle/dz = Q_b/K$ comes from	26
5.2.2	The rectified flux u_{rect}	26
5.3	The method: alternate “settle the skin” and “rebuild the deep”	26
5.4	Why it’s both fast <i>and</i> unbiased	27
5.5	“Flux closure” — how the code proves it worked	28
6	The K_d retrieval and uncertainty analysis	28
6.1	The data: turning raw HFE records into “equilibrium temperatures”	28
6.2	The retrieval: sweep K_d , minimize RMSE	28
6.3	Uncertainty: the bootstrap	29
6.4	Inter-site contrast and the K_d – Q_b degeneracy	30
6.5	The optional bedrock toggle (for future ice work)	31
7	Code tour: every file, in plain terms	31
7.1	The <code>lunar/</code> package (the engine)	31
7.2	<code>scripts/pipeline/</code> (compute results → write <code>output/*.json</code>)	32
7.3	<code>scripts/figures/</code> and <code>scripts/analysis/</code>	33
7.4	The end-to-end data flow	33
8	How to run it yourself	34
8.1	One-time setup	34
8.2	Run the tests first (always do this)	34
8.3	Verify the committed headline numbers (no computation needed)	35
8.4	Reproduce the science (recompute from scratch)	35
8.5	Run a single illuminating script	35
8.6	The notebooks (guided, in order)	36
8.7	Where outputs land	36
9	Glossary	36
10	A learning roadmap: what to study next	38

List of Figures

1	Conductivity rises with depth and temperature	15
2	The daily heat wave dies out with depth	16
3	Below the skin the mean temperature keeps rising	17
4	Sun–Moon–surface geometry. At a site at latitude ϕ , the Sun is not overhead at “noon” unless $\phi = 0$. The solar zenith angle $\theta_\odot$ sets how much solar power hits each square metre of horizontal ground (cosine law). The retrieval uses an idealised cycle $\cos \theta_\odot(t) = \cos \phi \cos(2\pi t/P)$; full SPICE ephemeris lives in <code>lunar/ephem.py</code> but is not required for the K_d^* fit.	19
5	Idealised insolation $F_\odot(t)$ for northern mid-latitude ($\cos \phi \approx 0.90$). Sunlit half of the lunation: flux follows a cosine; night: clipped to zero. Peak is below S_0 because the Sun never reaches the zenith at $\phi > 0$. Code: <code>insol = S0 * cos_lat * np.maximum(0, np.cos(phase))</code> in <code>retrieve_kd.py</code>	20
6	Top boundary control volume. Absorbed sunlight (after albedo A) must equal the sum of infrared emission and heat conducted into the regolith. T_s is not stored as a fixed number — it is solved each hour so the balance holds. This is what <code>_solve_surface_newton</code> does in <code>solver.py</code>	20
7	Bottom boundary. A small steady flux from the Moon’s interior. No orbital variation — only the top sees the Sun.	21
8	The geometric depth grid	22
9	Numba speed-up	25
10	The two-clock Anchor Point Method	27
11	The K_d retrieval as an RMSE sweep	29
12	Bootstrap distributions of the retrieved K_d^*	30
13	The phase-2 deep-temperature diagnostic	33
14	The end-to-end data flow	34

How to read this guide

Read it top to bottom, once, slowly. Each section builds on the one before. Wherever you see a file path like `lunar/solver.py` or a function name like `solve_periodic_equilibrium`, that is a *real thing* in the repository you can open and look at. The guide is the map; the code is the territory. The coloured boxes are signposts:

- **In plain English** — the jargon-free version.
- **Analogy** — an everyday picture to lean on.
- **Key idea** — the one thing to remember.
- **Try it yourself** — a small action at the keyboard.
- **Heads-up** — a subtlety or common confusion.

1 The big picture: what question does this repo answer, and why care?

1.1 The scientific question in one breath

How well does the loose, dusty soil on the Moon (called **regolith**) conduct heat, about a metre below the surface, *at two specific places* — the Apollo 15 and Apollo 17 landing sites? And is that number *the same at both sites*, or different?

The single number that captures “how well does deep Moon soil conduct heat” is called K_d — the *deep thermal conductivity*. This whole repository exists to measure K_d at those two sites and to ask whether they differ.

1.2 Why this is hard, and why it matters

- The Moon has been mapped from orbit in incredible detail. But orbiters only see the **surface**. To know what is happening a metre *down*, you need a thermometer *in the ground*.
- Thermometers have been buried in the Moon **exactly twice in history**: during Apollo 15 (1971) and Apollo 17 (1972). Astronauts drilled holes and dropped temperature probes down them. This was the **Heat-Flow Experiment (HFE)**. The instruments radioed back temperatures from 1971 to 1977.
- That 1971–1977 record is therefore *the only direct measurement of metre-scale lunar soil conductivity that exists*. Everything else is an educated guess extrapolated from surface data.
- Current models (notably the widely used **Hayne et al. 2017** model) apply a **single conductivity value to the entire Moon**. This project asks: is one global number actually good enough, or do different places genuinely differ?

We have two underground thermometers’ worth of data, ever, from the whole Moon. This project squeezes that scarce data as hard as it can to answer one sharp question: does deep Moon soil conduct heat the same everywhere, or not?

Why anyone cares.

- Future missions want to detect **water ice** trapped below the lunar surface. Whether ice stays frozen or sublimates away depends on the **temperature profile underground**, which depends directly on K_d .
- An upcoming terahertz mission (**TSUKIMI**) will try to “see” into the regolith from orbit. To interpret what it sees, it needs the underground temperature-versus-depth curve $T(z)$ as a boundary condition — and that curve is set by K_d .
- More fundamentally: if “one number for the whole Moon” is wrong, every model built on it inherits the error.

1.3 The headline result (the answer)

After all the modelling and statistics, this is what the repository finds (these exact numbers are committed in `output/kd_retrieval_results.json` and quoted in the paper abstract).

- The little $*$ in K_d^* means “the best-fit value” (the star is the retrieved estimate).

Table 1. The retrieved deep conductivity at each Apollo site.

Site	Retrieved K_d^*	Units
Apollo 15	4.58 (+ 1.33 / -0.28)	$\text{mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$
Apollo 17	8.12 (+ 0.49 / -0.61)	$\text{mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$

- The units $\text{mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ are milliwatts per metre per kelvin — a measure of conductivity (subsection 3.5 explains what conductivity units mean). For scale, the “one global value” everyone used before is $K_d = 3.4$ in the same units, window glass is ~ 1000 , and copper is $\sim 4 \times 10^8$. Lunar regolith is an *extraordinary* insulator.
- The **inter-site contrast** (the difference, A17 - A15) is $\approx 3.3 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$, with a 95% confidence interval of roughly $[0.4, 4.6]$ that **excludes zero** ($p \approx 0.01$). In plain terms: the two sites really do appear to differ — Apollo 17 soil conducts heat noticeably better than Apollo 15 soil — by roughly a factor of 1.8.

The two sites are different. Apollo 17 deep soil conducts heat about **1.8 times better** than Apollo 15 deep soil. That difference, established from the only underground data that exist, is the scientific contribution of the whole project. The rest of this guide explains *how* the repository gets there.

2 Notation reference — every symbol used in this project

This section is a **complete symbol map**. Keep it open while reading the physics sections or the code. Symbols are grouped by topic; each entry gives the meaning, typical units, and where it appears in the repository.

2.1 Coordinates, time, and indexing

z	Depth, positive downward from the regolith surface	m	<code>grid.z_mid</code> , paper $z = 0$ at surface
t	Time within one lunation cycle	s	<code>PixelInputs.t</code> , hourly steps in production
$T(z, t)$	Temperature at depth z and time t	K	<code>PixelOutputs.T[i,k]</code>
$T_s(t)$	Skin (surface) temperature	K	<code>PixelOutputs.T_surface</code> , solved by Newton each step
$\langle T \rangle(z)$	Cycle- mean (time-averaged) temperature at depth z	K	<code>EquilibriumResult.T_mean</code> , “annual mean” in paper
$\partial T / \partial t$	Rate of temperature change in time	K/s	discretised as $(T^{n+1} - T^n) / \Delta t$
$\partial T / \partial z$	Temperature gradient with depth	K/m	<code>np.gradient</code> , Fourier flux
i, k	Grid index (depth), time index	—	cell i , timestep k in <code>PixelOutputs.T[i,k]</code>

2.2 Heat, flux, and energy

q	Heat flux — thermal power crossing a horizontal plane per unit area	W/m ²	Fourier: $q = -K \partial T / \partial z$
Q_b	Basal / geothermal heat flux — steady upward flux from the lunar interior at the column bottom	W/m ²	<code>SITES[...]['Q_BASAL']</code> ; $A_{15} = 0.021$, $A_{17} = 0.015$
u_{rect}	Rectified (eddy) flux — extra cycle-mean flux in the diurnal skin from $K(T)$ nonlinearity	W/m ²	<code>_rectified_flux</code> in <code>equilibrium.py</code>
ρ	Bulk density of regolith	kg/m ³	<code>density_hayne</code>
c_p	Specific heat — energy to warm 1 kg by 1 K	J · kg ⁻¹ · K ⁻¹	<code>specific_heat</code>
ρc_p	Volumetric heat capacity (thermal inertia per volume)	J · m ⁻³ · K ⁻¹	product <code>cap</code> in <code>_step</code>

2.3 Thermal conductivity model (Hayne 2017)

$K(T, z)$	Total effective thermal conductivity	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$	<code>conductivity_hayne</code>
$K_c(z)$	Structural (contact) part of K	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$	rises from K_s to K_d with depth
K_s	Surface contact conductivity	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$	7.4×10^{-4} (<code>K_SURFACE</code>)
K_d	Deep contact conductivity — the retrieved parameter	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$	A15 $K_d^* = 4.58$, A17 = $8.12 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$
H	Compaction scale height for $K_c(z)$ and $\rho(z)$	m	~ 0.06 (<code>H_PARAMETER</code>)
χ	Radiative conductivity coefficient	—	2.7 (<code>CHI_RADIATIVE</code>)
T_{ref}	Reference temperature in radiative term	K	350 in code
ρ_s, ρ_d	Surface / deep bulk density	kg/m^3	~ 1100 / ~ 1800

2.4 Surface radiation, albedo, and orbital forcing

S_0	Solar constant at 1 AU (flux at top of atmosphere)	W/m^2	1361 (<code>S0</code> in <code>config.py</code>)
$F_{\odot}(t), S(t)$	Insolation — solar irradiance on local horizontal surface	W/m^2	<code>insolation</code> array in <code>run_with</code>
$\theta_{\odot}(t)$	Solar zenith angle — angle from local vertical to Sun	rad or $^{\circ}$	idealised: $\cos \theta_{\odot} = \cos \phi \cos(2\pi t/P)$
ϕ	Site latitude (north positive)	$^{\circ}$	A15 26.13°N , A17 20.19°N
P	Mean synodic lunation (Sun $\rightarrow$ Sun as seen from site)	s	2 551 443 (<code>T_LUNAR</code>)
A	Bond albedo — fraction of sunlight reflected	—	~ 0.13 per site
$(1 - A)$	Fraction of sunlight absorbed at surface	—	~ 0.87
ε	Emissivity — efficiency of thermal radiation	—	0.95
σ	Stefan–Boltzmann constant	$\text{W} \cdot \text{m}^{-2} \cdot \text{K}^{-4}$	5.6704×10^{-8} (<code>SIGMA_SB</code>)
$\varepsilon \sigma T_s^4$	Outgoing thermal (infrared) radiative flux	W/m^2	surface energy balance

2.5 Boundary conditions and problem setup

Eq. (5)	Interior heat equation (PDE)	all interior cells
Eq. (11)	Top BC: radiative surface energy balance	surface_energy_balance_residual
Eq. (12)	Bottom BC: prescribed basal flux Q_b	d[-1] += dt*Q_b/cap[-1]
$z_{\max}$	Bottom of model column	5 m (GRID['z_max'])
z_{anchor}	Anchor depth for equilibrium method	0.55 m

2.6 Skin depth, diffusivity, and diurnal damping

α	Thermal diffusivity	$\alpha = K/(\rho c_p)$
ω	Angular frequency of diurnal forcing	$\omega = 2\pi/P$
δ	Thermal skin depth	$\delta = \sqrt{2\alpha/\omega}$; few cm

2.7 Numerical discretisation

$\Delta z_i, dz$	Thickness of layer i (geometric grid)	2 mm at top, grows $\sim 8\%$ /layer
Δt	Timestep	1 h (DT_STEP)
N_z	Number of depth cells	~ 69 for 5 m column
N_t	Timesteps per lunation	709

2.8 Retrieval, data, and statistics

K_d^*	Retrieved deep conductivity minimising RMSE	kd_retrieval_results.json
$T_{\text{eq},i}$	Equilibrium temperature at sensor i (stable window mean)	extract_sensor_stability
R_i	Residual $T_{\text{model}} - T_{\text{eq}}$ at sensor i	run_kd_sweep_extended
RMSE	Root mean square error of residuals	kd_star_from_residuals

Sign conventions used throughout. Depth z increases **downward**. Heat flux q is positive when flowing **downward** (Fourier: $q = -K \partial T / \partial z$). Basal flux Q_b is positive when

flowing **upward** from the interior — so at the bottom boundary $-K \partial T / \partial z = Q_b$ implies $\partial T / \partial z > 0$ (warmer with depth). Temperature is always in **kelvin** (K), never Celsius, in the solver.

3 Physics fundamentals, from scratch

This section assumes you know nothing about heat physics. Take it slowly.

3.1 Heat vs. temperature (they are not the same thing)

- **Temperature** is *how hot something is* — a single number at a point, like 300 K. It tells you the average jiggling energy of the atoms there.
- **Heat** is *energy in transit* — energy flowing from a hotter place to a colder place. You measure it as a *rate of flow*.

Temperature is like the **water level** in a tank; heat is like the **water flowing** between tanks. Two tanks at the same level (same temperature) have no flow between them, even if one is huge and one is tiny.

Kelvin (K) is just the temperature scale scientists use. It starts at absolute zero (the coldest possible), so there are no negative numbers. $0^\circ\text{C} = 273.15\text{ K}$. Lunar daytime soil-surface can hit $\sim 390\text{ K}$ (scorching); lunar night can fall below 100 K (-170°C). A metre down, it settles to a near-constant $\sim 250\text{ K}$.

3.2 Conduction: how heat moves through a solid

There are three ways heat travels: **conduction** (through direct contact, atom bumping atom), **convection** (carried by a moving fluid like air or water), and **radiation** (electromagnetic waves, like sunlight or the warmth you feel from a fire).

The Moon has **no air and no water underground**, so inside the regolith column there is essentially only **conduction** (plus a small radiative effect across the tiny gaps between dust grains, which we fold into an “effective” conductivity — see [subsection 3.5](#)). This is why the model is “1-D conduction.”

The basic law of conduction (**Fourier’s law**) says, in words: *heat flows from hot to cold, and the rate of flow is proportional to how steep the temperature change is, times how good a conductor the material is*. As a tiny equation,

$$q = -K \frac{dT}{dz}, \tag{1}$$

where:

- dT/dz is the **temperature gradient**: how fast temperature changes as you go down (z is depth). Steep gradient $\rightarrow$ fast flow.
- K is the **thermal conductivity**: the material property — high K means heat flows easily (metal), low K means it resists (styrofoam, lunar dust).
- The minus sign just means heat flows *down* the gradient, from hot to cold.

In this repo Fourier’s law literally appears as the conductive term in the surface energy balance and in every interior step. See `surface_energy_balance_residual` in `lunar/solver.py`.

3.3 The 1-D heat equation — full derivation from scratch

This section answers three questions in order: (1) what problem is the code actually solving? (2) where does the differential equation come from, step by step, with no prior PDE course assumed? (3) what do the top and bottom boundaries mean? The final equation is the one in `lunar/solver.py` and in the paper (Eq. (5)).

3.3.1 The problem in one sentence

Given how sunlight hits the lunar surface hour by hour, and given how the soil conducts heat at each depth, **find** the temperature T at every depth z and every time t .

The simulator predicts a **temperature profile through the soil column** — temperatures from the surface down to 5 m — and updates that profile as the lunar day/night cycle rolls on. In code, the answer lives in `PixelOutputs.T` from `solve_pixel` in `lunar/solver.py`: a 2-D array of shape (N_z, N_t) .

3.3.2 Picture a thin horizontal slice of soil

Imagine the regolith as a **stack of thin horizontal pancakes**, each between depth z and $z + \Delta z$. Focus on one pancake in the middle:

```

surface (z = 0)  <- sunlight, radiation to space
-----
warmer layer
-----
OUR SLICE  (thickness Delta z)  <- top face (depth z)
-----
cooler layer  <- bottom face (z + Delta z)
-----
...
-----
bottom (z = 5 m) <- steady heat from interior

```

We track a column of soil $1\text{ m} \times 1\text{ m}$ in footprint. The slice has mass $\rho \Delta z$ per m^2 and thermal mass $\rho c_p \Delta z$ — energy needed to warm it by 1 K. The product ρc_p is **thermal inertia**: dense, high- c_p soil is hard to warm or cool (like a heavy cast-iron pan).

3.3.3 Heat flux at each face

Heat flux q [$\text{W} \cdot \text{m}^{-2}$] is the rate thermal energy crosses a horizontal plane. Fourier’s law (Eq. (1)) gives

$$q(z) = -K(T, z) \frac{\partial T}{\partial z}. \quad (2)$$

Heat flows from hot toward cold. For a slice between z and $z + \Delta z$, let q_{in} be flux across the top face and q_{out} across the bottom.

3.3.4 Energy conservation (the bookkeeping step)

Energy is not created or destroyed in the slice:

$$\underbrace{\rho c_p \Delta z \frac{\partial T}{\partial t}}_{\text{rate stored energy changes}} = q_{\text{in}} - q_{\text{out}}. \quad (3)$$

The left side is how fast the slice warms; the right side is net heat flowing in.

3.3.5 From two faces to a derivative

For a thin slice, $q_{\text{out}} - q_{\text{in}} \approx (\partial q / \partial z) \Delta z$, so $q_{\text{in}} - q_{\text{out}} \approx -(\partial q / \partial z) \Delta z$. Substitute into Eq. (3) and cancel Δz :

$$\rho c_p \frac{\partial T}{\partial t} = - \frac{\partial q}{\partial z}. \quad (4)$$

Insert $q = -K \partial T / \partial z$:

$$\rho(z) c_p(T) \frac{\partial T}{\partial t} = \frac{\partial}{\partial z} \left[K(T, z) \frac{\partial T}{\partial z} \right]. \quad (5)$$

That is the **1-D heat equation** — the single rule the whole simulator enforces.

Why “partial” derivatives? T depends on *both* depth z *and* time t . $\partial T / \partial t$ means “how fast temperature at this depth is changing right now”; $\partial T / \partial z$ means “how steep the profile is at this instant.” You do not need a full PDE course to use this — treat them as separate rates of change.

Discrete version (what the computer does). On a finite grid,

$$(\rho c_p)_i \frac{T_i^{n+1} - T_i^n}{\Delta t} = (\text{flux in from above}) - (\text{flux out below}).$$

Crank–Nicolson ([subsection 4.3](#)) rearranges this so $\Delta t = 1$ h is stable. Inside `_step` in `lunar/solver.py`: `cap_i * dT_i/dt = flux_right(i) - flux_left(i)`.

3.3.6 Reading the final equation, term by term

Piece	Plain English	Code
$T(z, t)$	Temperature at depth z , time t	<code>PixelOutputs.T[i,k]</code>
$\partial T / \partial t$	Warming/cooling rate	finite difference over Δt
$\rho(z), c_p(T)$	Density, specific heat	<code>rho_func, cp_func</code>
ρc_p	Thermal inertia	<code>cap</code> in <code>_step</code>
$K(T, z)$	Conductivity	<code>K_func</code>
$K \partial T / \partial z$	Heat flux (Fourier)	face fluxes in <code>_step</code>
$\partial / \partial z [\dots]$	Net flux in minus out	difference of face fluxes

Left side = storage; **right side** = net conduction from neighbours. When K varies with z or T , it stays *inside* the $\partial / \partial z$ — the same gradient can carry different flux where conductivity changes.

3.3.7 The two boundaries (summary)

The PDE needs two boundary conditions. [subsection 3.8](#) derives both from scratch and explains their connection to the interior equation and to `lunar/solver.py`. In brief: top radiative balance (Eq. (11)); bottom geothermal flux (Eq. (12)).

3.3.8 What “solving” means in this repository

The simulator solves an **initial-boundary-value problem**:

1. **PDE** (interior): Eq. (5).
2. **Top BC** (each timestep): Eq. (11).
3. **Bottom BC**: Eq. (12).
4. **Initial condition**: starting profile $T(z, t=0)$ — the equilibrium driver (section 5) removes dependence on the guess.

One timestep in `solve_pixel`: evaluate K, ρ, c_p ; assemble Crank–Nicolson; Newton solve for T_s ; tridiagonal solve (`_thomas`) for interior cells; advance by Δt . Repeat ~ 709 steps per lunation.

Sanity checks. (i) Flat $T \Rightarrow$ zero flux $\Rightarrow \partial T / \partial t = 0$. (ii) Steady state with bottom heat $\Rightarrow d\langle T \rangle / dz = Q_b / K$ (geothermal gradient). (iii) Larger $K \Rightarrow$ deeper diurnal damping (larger skin depth). These are exactly what the code exhibits.

“1-D” means we only track depth (z), ignoring sideways variation — valid over the small patch around a borehole.

3.4 The three soil properties — what they mean and why the heat equation needs them

3.4.1 Thermal capacity: why ρc_p sits on the left

When you warm soil, you store **internal energy**. For a slice of thickness Δz and footprint 1 m^2 :

$$\text{mass} = \rho \Delta z \quad (\text{kg/m}^2), \quad \text{energy to warm by } 1 \text{ K} = \rho c_p \Delta z \quad (\text{J/m}^2).$$

So ρc_p is **volumetric heat capacity** ($\text{J} \cdot \text{m}^{-3} \cdot \text{K}^{-1}$). Dividing the energy-balance rate by this capacity gives the warming rate — exactly why ρ and c_p multiply on the **left** of Eq. (5).

Symbol	Name	Units	Code
$\rho(z)$	Density	kg/m^3	<code>density_hayne</code>
$c_p(T)$	Specific heat	$\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$	<code>specific_heat</code>
ρc_p	Volumetric heat capacity	$\text{J} \cdot \text{m}^{-3} \cdot \text{K}^{-1}$	<code>cap in _step</code>
$K(T, z)$	Conductivity	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$	<code>conductivity_hayne</code>

ρc_p is the soil’s **thermal mass** (hard to change temperature). K is its **conductance** (how easily heat travels through).

All three are evaluated at every grid cell from `lunar/properties.py`.

3.5 Thermal conductivity $K(T, z)$ — full physical picture

K_d — the quantity this project retrieves — lives inside the conductivity formula. Two channels carry heat in vacuum regolith:

1. **Contact conduction** — grain-to-grain; weak when fluffy.

2. Radiative conduction across vacuum gaps — grows with T^3 .

Hayne splits these explicitly:

$$K(T, z) = \underbrace{K_c(z)}_{\text{structural}} \times \underbrace{[1 + \chi (T/350)^3]}_{\text{radiative boost}}.$$

3.5.1 Structural part $K_c(z)$ — compaction with depth

$$K_c(z) = K_d - (K_d - K_s) e^{-z/H}. \quad (6)$$

At $z=0$: $K_c=K_s$ (fluffy surface, $\sim 0.74 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$). Deep down: $K_c \rightarrow K_d$ (the unknown we fit). The scale height $H \sim 6 \text{ cm}$ sets how fast the transition happens; at $z=H$ the rise is $\sim 63\%$ complete.

3.5.2 Radiative boost — why T^3 ?

Across a vacuum gap, net radiative flux $\propto T_1^4 - T_2^4 \approx 4T^3 \Delta T$ for small ΔT . Hayne folds this into:

$$K(T, z) = K_c(z) [1 + \chi (T/350)^3]. \quad (7)$$

At $T \approx 250 \text{ K}$ the factor is ~ 2 — radiation roughly **doubles** effective K . **Not optional** (known bug if omitted).

3.5.3 Density and specific heat

Density compacts with the same exponential form (Hayne Eq. 5): $\rho(z) = \rho_d - (\rho_d - \rho_s) e^{-z/H}$. Specific heat $c_p(T)$ is a 4th-order polynomial (`specific_heat`); it sets thermal inertia but is not the retrieval's free parameter.

So K_d is one knob in a fixed published shape; the retrieval freezes K_s , H , χ , and slides only K_d until the model matches Apollo data.

Listing 1. Cited conductivity constants in `lunar/constants.py`.

```
1 # Surface contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017) Table 2.
2 K_SURFACE: float = 7.4e-4
3 # Deep contact conductivity [W m^-1 K^-1] -- Hayne et al. (2017) Table 2.
4 K_DEEP: float = 3.4e-3
5 # Radiative conductivity coefficient chi [dimensionless] -- Hayne et al. (2017).
6 CHI_RADIATIVE: float = 2.7
```

Units. Conductivity is in $\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ (watts per metre per kelvin). $3.4 \times 10^{-3} \text{ W} \cdot \text{m}^{-1} \cdot \text{K}^{-1} = 3.4 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ — the paper quotes milliwatts (mW) because the numbers are tiny. $1 \text{ W} = 1000 \text{ mW}$. So the committed $K_d^* = 4.58 \times 10^{-3}$ in the JSON prints as $4.58 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$.

3.6 The thermal skin and skin depth — full derivation

3.6.1 Thermal diffusivity

$$\alpha = \frac{K}{\rho c_p} \quad [\text{m}^2/\text{s}].$$

Dividing the heat equation by ρc_p (constant properties): $\partial T / \partial t = \alpha \partial^2 T / \partial z^2$ — a diffusion equation.

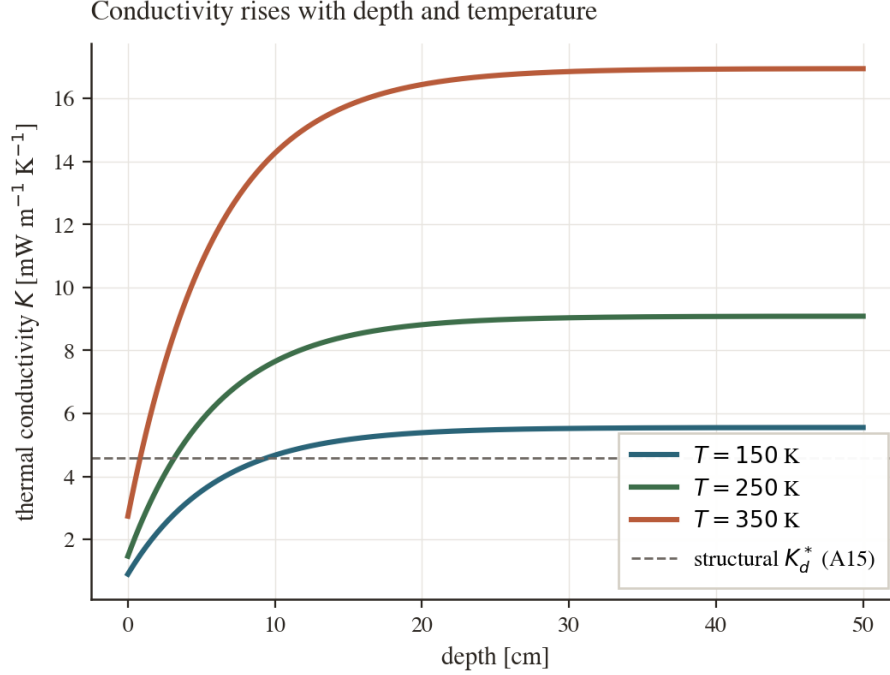


Figure 1. The Hayne $K(T, z)$ law, Eqs. (6)–(7), at the Apollo 15 K_d^* . **What to notice:** each curve climbs steeply over the first few centimetres — that is the structural rise from fluffy surface (K_s) toward the deep value K_d governed by the scale height H . The three curves (cold to hot) are spread apart by the radiative T^3 boost: hotter soil conducts *better* even at the same depth. **Why it matters:** the single deep number we retrieve, K_d^* , is the plateau these curves are climbing toward.

3.6.2 The decaying thermal wave

For sinusoidal surface forcing $T(0, t) = T_{\text{mean}} + A_s \cos(\omega t)$:

$$T(z, t) = T_{\text{mean}} + A_s e^{-z/\delta} \cos(\omega t - z/\delta).$$

At $z = \delta$ the amplitude is $A_s/e \approx 0.37A_s$.

3.6.3 Deriving the skin depth

Substituting into the diffusion equation gives

$$\delta = \sqrt{\frac{2\alpha}{\omega}}. \quad (8)$$

Larger $\alpha \Rightarrow$ deeper skin; larger $\omega \Rightarrow$ shallower skin. For the Moon, $\delta \sim 6$ cm.

Apollo sensors at 0.8 m–2.4 m sit **10–40× deeper** than δ — below the diurnal noise, in the quiet zone set by the geothermal gradient. This underpins MIN_DEPTH_CM = 80 and the Anchor Point Method.

3.7 Radiation, sunlight, and the geothermal gradient — full picture

3.7.1 Stefan–Boltzmann: why T^4 ?

Every surface emits radiation $q_{\text{rad}} = \varepsilon \sigma T^4$. A slightly hotter surface radiates vastly more ($2\times$ temperature $\Rightarrow 16\times$ power). This makes the surface balance non-linear (Newton solve, [subsection 4.5](#)).

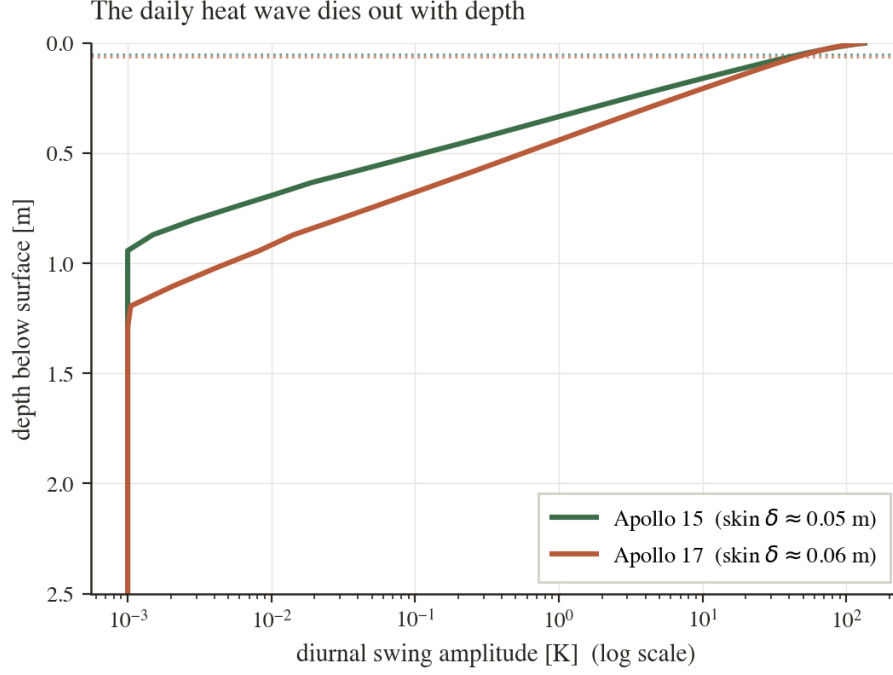


Figure 2. The size of the day/night temperature swing as you go down, for both sites (note the *logarithmic* horizontal axis). **What to notice:** the swing starts at ~ 100 K at the surface and collapses by a factor of a thousand within about a metre — the dotted line marks the $1/e$ skin depth δ , only a few centimetres. Below that the curve plunges to essentially nothing. **Why it matters:** the Apollo sensors used for the fit all sit well below this skin, in the quiet zone where temperature is set purely by the deep heat flow.

3.7.2 Surface energy balance

$$\underbrace{(1 - A) S(t)}_{\text{absorbed}} - \underbrace{\varepsilon \sigma T_s^4}_{\text{radiated}} - \underbrace{q_{\text{cond}}(0)}_{\text{into soil}} = 0. \quad (9)$$

Implemented in `surface_energy_balance_residual` (`lunar/solver.py`).

3.7.3 Insolation model

$$S(t) = S_0 \cos(\text{lat}) \max(0, \cos(2\pi t/T_{\text{lunation}})).$$

Built in `run_with`. Idealised diurnal cosine; neglected terms are bounded in the paper.

3.7.4 Geothermal gradient — full derivation

Bottom BC: $-K \partial T / \partial z|_{\text{bottom}} = Q_b$. Time-average the heat equation in periodic steady state ($\partial \langle T \rangle / \partial t = 0$):

$$\frac{\partial}{\partial z} \left[K \frac{\partial \langle T \rangle}{\partial z} \right] = 0 \quad \Rightarrow \quad K \frac{d \langle T \rangle}{dz} = \text{const} = Q_b.$$

Hence

$$\frac{d \langle T \rangle}{dz} = \frac{Q_b}{K}. \quad (10)$$

Larger Q_b or smaller $K \Rightarrow$ steeper rising mean profile. Example: $Q_b = 0.021$ W/m², $K_d \approx 0.0046$ W/m/K $\Rightarrow$ gradient ≈ 4.6 K/m.

This Q_b matters for the retrieval: deep T depends on the **ratio** Q_b/K_d — the K_d - Q_b **degeneracy** (subsection 6.4).

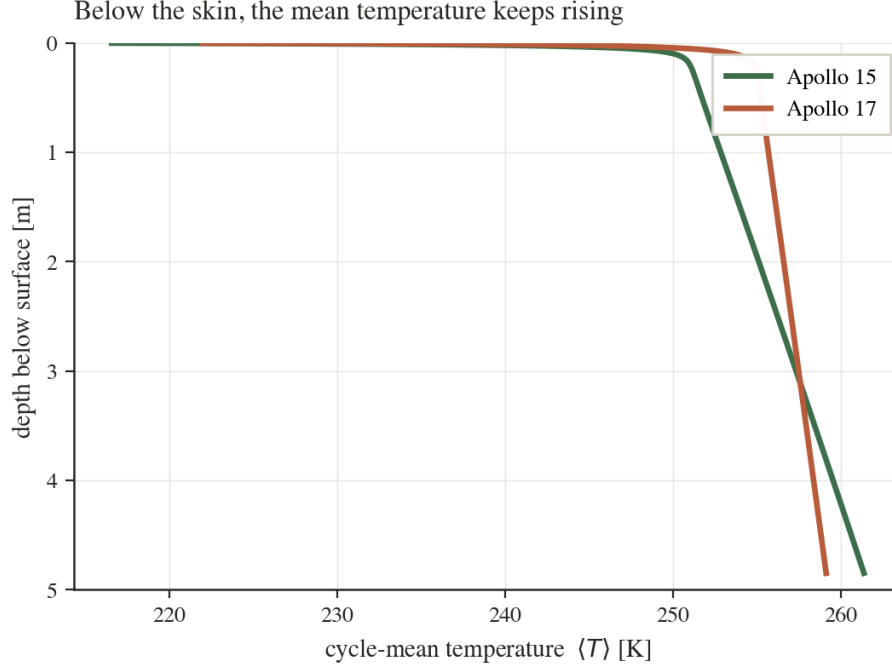


Figure 3. The cycle-averaged temperature $\langle T \rangle$ versus depth for both sites, from the certified equilibrium solver. **What to notice:** below ~ 1 m the curve becomes a straight, *tilted* line — it does not flatten out. That tilt is the geothermal gradient Q_b/K of Eq. (10). **Why it matters:** the deep slope encodes the ratio Q_b/K_d ; reading the Apollo temperatures against this line is exactly how the project pins down K_d .

3.8 Boundary conditions — full derivation and connection to the main equation

3.8.1 Why the main equation is not enough

The heat equation (Eq. (5)) is second-order in z , so it needs **two boundary conditions** plus an initial condition $T(z, t=0)$ to pick a unique solution. The complete problem: interior PDE + top BC + bottom BC + initial profile (paper Eqs. 1–3).

3.8.2 One unifying idea: energy balance at every control volume

Interior cells: flux in minus flux out equals storage. At boundaries the same bookkeeping applies, but one face is the domain edge (space above, or deep interior below) — not another soil cell.

3.8.3 Top boundary — radiative surface balance

At $z = 0$ there is no conduction from above. Energy balance on the surface skin:

$$(1 - A) S(t) - \varepsilon \sigma T_s^4 - q_{\text{cond}}(0) = 0. \quad (11)$$

The conductive term is Fourier’s law at the top face: $q_{\text{cond}}(0) = -K \partial T / \partial z|_{0+} \approx K_{\text{surf}}(T_s - T_{\text{sub}})/(\Delta z_0/2)$, implemented in `surface_energy_balance_residual`. Each timestep, `_solve_surface_newton` finds T_s ; the first row of the tridiagonal system uses T_s as a ghost temperature coupling the BC to the interior PDE.

3.8.4 Bottom boundary — geothermal flux

Below the 5 m column we prescribe upward flux Q_b (Neumann condition):

$$-K \left. \frac{\partial T}{\partial z} \right|_{z=z_{\max}} = Q_b. \quad (12)$$

For the deepest cell, the outgoing flux at the bottom face is fixed to Q_b rather than computed from a neighbour. In `_step: d[-1] += dt * Q_b / cap[-1]` adds the basal energy source. In steady state this forces $K d\langle T \rangle / dz = Q_b$ everywhere (Eq. (10)).

3.8.5 How the pieces fit in one timestep

1. **Top:** Newton solve for T_s from radiative balance.
2. **Interior:** Crank–Nicolson discretisation of Eq. (5).
3. **Bottom:** add Q_b source to the last cell.
4. **Coupled solve:** one Thomas tridiagonal sweep for T^{n+1} .

The BCs are not separate from the PDE — they are the same energy-conservation principle at the domain edges, wired into the same linear system.

Sanity check. Day/night surface swings $\leftarrow$ top BC; rising mean profile $\leftarrow$ bottom BC; quiet deep sensors $\leftarrow$ interior damping (skin depth). Wrong BCs would still run but would not match Apollo or Diviner.

3.9 Orbital mechanics and the top boundary — full picture with diagrams

The **top boundary** is where the Moon meets space. Unlike Earth, there is no atmosphere to scatter or absorb sunlight, and no wind to carry heat away. Everything that heats or cools the surface comes from:

1. **Direct sunlight** (orbital geometry sets how strong it is at each moment),
2. **Thermal radiation** the surface emits to the cold sky,
3. **Conduction** into the regolith below.

This subsection explains the **orbital part** (where does $F_{\odot}(t)$ come from?) and ties it to the **surface energy balance** that closes the top BC.

3.9.1 The Moon in orbit: what “one day” means

The Moon rotates once per orbit around Earth, so the same face always points toward Earth. From a point on the lunar surface, “day” and “night” are set by the Sun, not by watching Earth.

- **Synodic month** $P \approx 29.53$ days: time from one local noon (Sun highest) to the next. This is the period used in the code ($T_{\text{LUNAR}} \approx 2.55 \times 10^6$ s).
- **Subsolar point:** the spot on the Moon where the Sun is directly overhead ($\theta_{\odot} = 0$). It marches around the equator once per synodic month.
- **Site latitude** ϕ : Apollo 15 is at 26.1°N , Apollo 17 at 20.2°N . The Sun never passes directly overhead there; at local “noon” the Sun is lower in the sky than at the equator.

Imagine standing on the Moon at an Apollo site. The Sun rises, tracks across the sky at a height that depends on your latitude, sets, and you wait through a long night (~ 14 Earth days) until it rises again. The model does not simulate every crater shadow or mountain — it uses a smooth “average day” for that latitude.

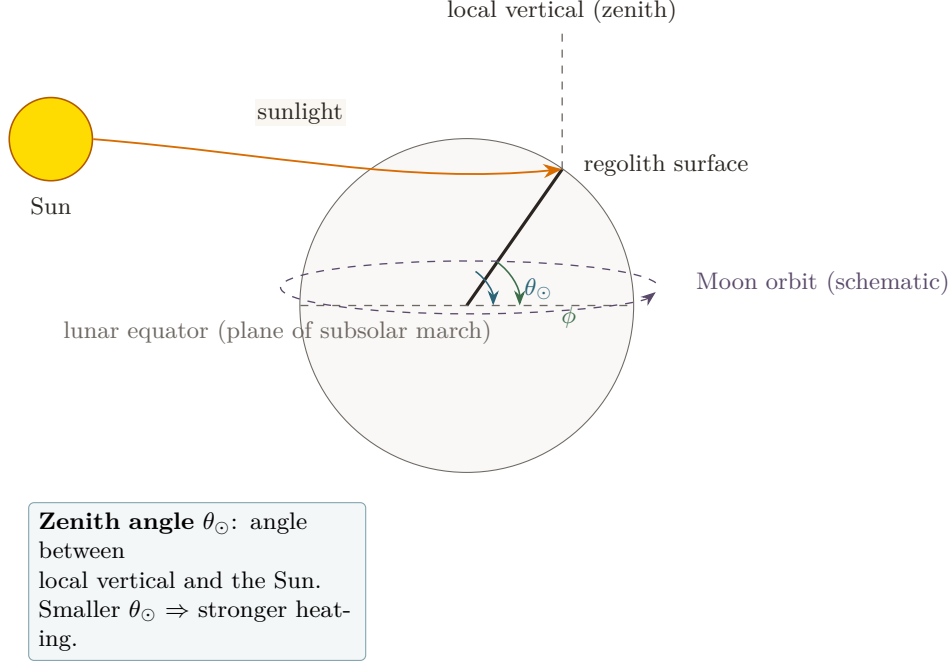


Figure 4. Sun–Moon–surface geometry. At a site at latitude ϕ , the Sun is not overhead at “noon” unless $\phi = 0$. The **solar zenith angle** $\theta_{\odot}$ sets how much solar power hits each square metre of horizontal ground (cosine law). The retrieval uses an idealised cycle $\cos \theta_{\odot}(t) = \cos \phi \cos(2\pi t/P)$; full SPICE ephemeris lives in `lunar/ephem.py` but is not required for the K_d^* fit.

3.9.2 From geometry to insolation: the cosine law

The solar constant $S_0 \approx 1361 \text{ W/m}^2$ is the flux in a beam perpendicular to the Sun. On a **horizontal** patch of ground, only the component normal to the beam counts:

$$F_{\odot}(t) = \max(0, S_0 \cos \theta_{\odot}(t)). \quad (13)$$

When the Sun is low ($\theta_{\odot}$ large), $\cos \theta_{\odot}$ is small and heating is weak. At night, $\cos \theta_{\odot} < 0$ and the $\max(0, \cdot)$ clips insolation to zero.

Idealised synodic model used in production. For each site latitude ϕ ,

$$\cos \theta_{\odot}(t) = \cos \phi \cos\left(\frac{2\pi t}{P}\right), \quad (14)$$

with $P = \text{T_LUNAR}$. This is the same formula as in the manuscript (Eq. (13)). It assumes:

- smooth variation over one lunation (no libration jitter at the hour scale),
- no solar **declination** tilt ($\pm 1.5^\circ$ neglected),
- no orbital **eccentricity** changing S_0 ($< 0.1\%$ effect on cycle mean).

Those corrections matter for *phase-resolved* work; this project compares **time-mean** temperatures over a stability window, so the simplified cycle is adequate (see manuscript §“Surface solar geometry”).

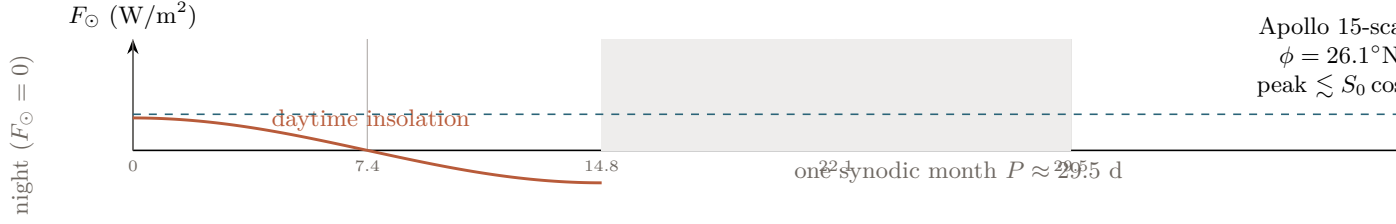


Figure 5. Idealised insolation $F_{\odot}(t)$ for northern mid-latitude ($\cos \phi \approx 0.90$). Sunlit half of the lunation: flux follows a cosine; night: clipped to zero. Peak is below S_0 because the Sun never reaches the zenith at $\phi > 0$. Code: `insol = S0 * cos_lat * np.maximum(0, np.cos(phase))` in `retrieve_kd.py`.

3.9.3 Top boundary: surface energy balance (control volume)

Take a thin control volume *at* the surface — the top face sees space, the bottom face touches the first regolith cell. In steady balance at each instant, Eq. (11) applies (derived in subsection 3.8), with insolation written explicitly as $F_{\odot}(t)$ instead of $S(t)$:

$$(1 - A) F_{\odot}(t) = \varepsilon \sigma T_s(t)^4 + q_{\text{cond}}(t).$$

Here $q_{\text{cond}} = -K \partial T / \partial z|_{z=0}$ is the downward conductive flux (positive when heat enters the ground).

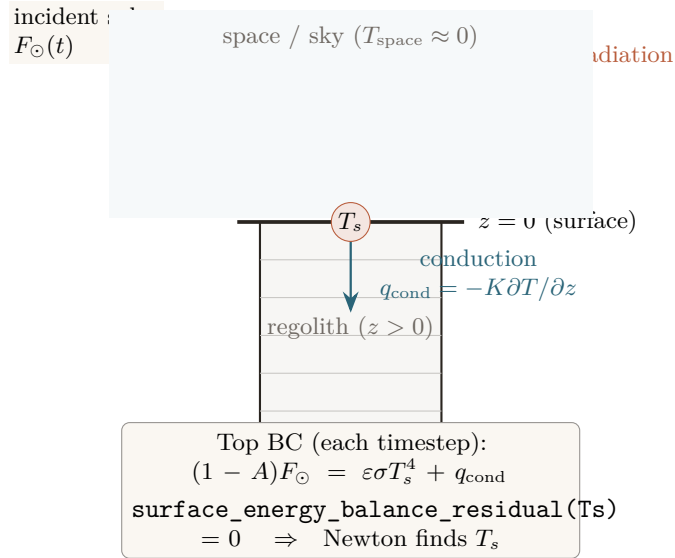


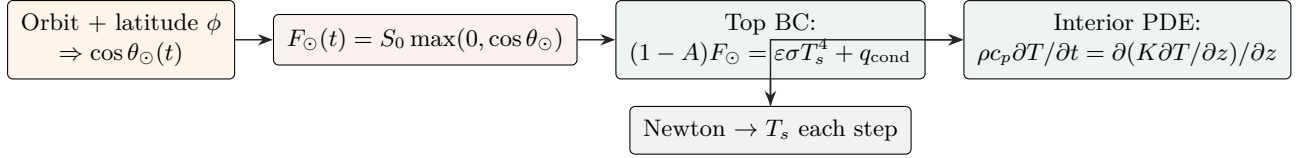
Figure 6. Top boundary control volume. Absorbed sunlight (after albedo A) must equal the sum of infrared emission and heat conducted into the regolith. T_s is not stored as a fixed number — it is **solved** each hour so the balance holds. This is what `_solve_surface_newton` does in `solver.py`.

The surface is like a pan on a stove that also glows in the dark. Sunlight pours in (when it’s day), the pan radiates heat to the cold kitchen (space), and the handle (regolith) conducts heat downward. At every instant the pan’s top temperature adjusts until those three flows

balance.

3.9.4 How orbital forcing connects to the heat equation

The chain from orbit to interior is:



Insolation is **prescribed** (known input from orbit model). Skin temperature T_s is **unknown** until the top BC is satisfied. Once T_s is known, it feeds the first interior cell and Crank–Nicolson advances $T(z, t)$ for all deeper layers.

3.9.5 Bottom boundary (for completeness)

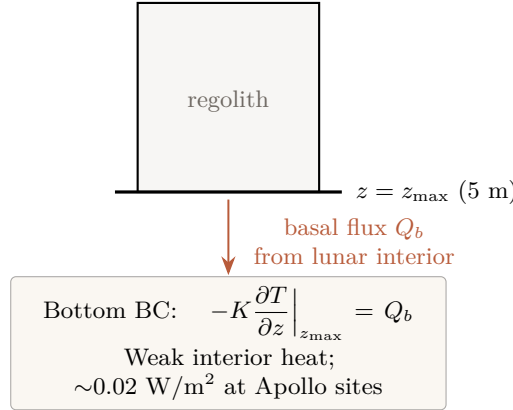


Figure 7. Bottom boundary. A small steady flux from the Moon’s interior. No orbital variation — only the top sees the Sun.

What is *not* in the top BC. No atmosphere, no wind, no latent heat (evaporation), no ground heat storage in a separate surface layer. The model is deliberately minimal: radiative balance + 1-D conduction. That matches airless-body practice for metre-scale regolith (Hayne et al. 2017; Vasavada et al. 2012).

3.9.6 Code map: from insolation to _step

For full SPICE-based zenith angles (sub-solar latitude, libration, exact timestamps), see `lunar/ephem.py`. The K_d^* retrieval does not require that path because it fits *mean* temperatures, not diurnal phase.

4 The numerical method, gently

The heat equation (subsection 3.3) is a continuous statement about every point and every instant. A computer can’t handle “infinitely many points.” So we **discretize**: chop space into finite

Step	Function / variable	Role
1	<code>cos_lat = np.cos(np.radians(lat))</code>	latitude factor
2	<code>phase = 2*np.pi*t/T_LUNAR</code>	lunation phase
3	<code>insol = S0 * cos_lat * max(0, cos(phase))</code>	build $F_{\odot}(t)$
4	<code>run_with(..., insolation=insol)</code>	pass into solver
5	<code>_solve_surface_newton(T_prev, insol_k, ...)</code>	top BC $\rightarrow T_s$
6	<code>_step(...)</code>	CN step for interior + bottom Q_b

Table 10. End-to-end path for orbital forcing and boundary conditions in the retrieval pipeline.

slices and time into finite steps, and turn the calculus into arithmetic the computer can repeat millions of times.

4.1 Slicing the depth: why a *geometric* grid

We split the 5-metre soil column into many thin horizontal layers and track one temperature per layer. The clever part: the layers are **not equally thick**.

- Near the surface, temperature swings violently over a few centimetres (the skin). To capture that we need **very thin** layers (~ 2 mm at the top).
- Deep down, temperature barely changes over a whole metre. Thin layers there would just waste computer effort.

So each layer is made $\sim 8\%$ thicker than the one above it. This is a **geometric grid** (each step multiplied by a constant ratio).

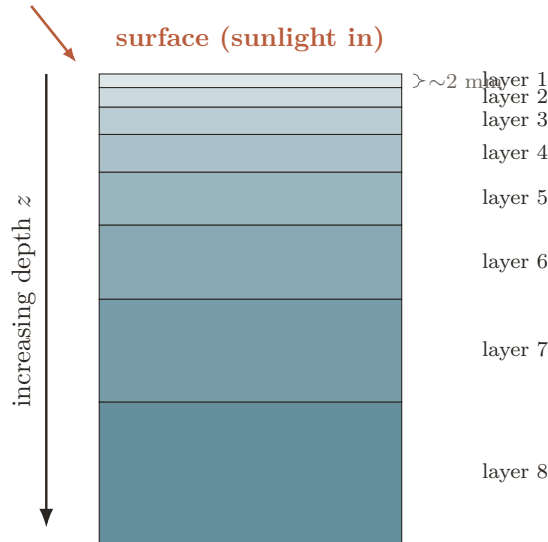


Figure 8. The geometric depth grid (thicknesses exaggerated). **What to notice:** slices are paper-thin at the top, where the daily heat wave is sharp, and grow $\sim 8\%$ thicker with each step downward, so the deep, sleepy soil is covered cheaply. **Why it matters:** this puts the computational effort exactly where the physics is fast-changing — like taking many photos per second of a sprint start but only one per minute of a marathon’s middle.

The grid docstring in `lunar/grid.py` puts it perfectly:

Listing 2. From the `lunar/grid.py` docstring.

```

1 Near the surface, where temperature swings wildly between lunar day
2 and night, we use paper-thin slices (~2 mm) so we capture the fast

```

```

3 changes. Deeper down, where temperature barely moves, the slices get
4 gradually thicker (each ~8% thicker than the one above). This
5 "geometric" spacing puts the computational effort where the action is.

```

The grid is built by `make_geometric_grid`, configured in `lunar/config.py`:

Listing 3. The production grid in `lunar/config.py`.

```

1 # --- depth grid (geometric) ---
2 GRID = dict(z_max=5.0, dz0=0.002, growth=0.08)

```

That reads: total depth 5 m, top layer 2 mm thick, each next layer 8% thicker (growth ratio 1.08). Uniform grids are *deliberately forbidden* in this project (the code even raises an error) because they under-resolve the skin while wasting effort deep down.

`lunar/constants.py` also defines *fallback* grid defaults (`Z_MAX_DEFAULT = 3.0`, `GROWTH_DEFAULT = 0.15`). Those are only used if someone calls `make_geometric_grid()` with no arguments. The real science runs always pass `GRID` from `config.py` (5 m / 2 mm / 1.08), so the production grid is the `config.py` one. Don't be confused by the two different numbers.

4.2 Stepping through time: what a “timestep” is

We also chop **time** into steps. Here each step Δt is **one hour** (`DT_STEP = 3600.0` seconds in `config.py`). Starting from the temperatures right now, we apply the heat equation to compute the temperatures one hour later, then repeat. One full lunar day (a “lunation”) is ~29.5 Earth days (2 551 443 s), so one simulated lunation is **709 hourly samples** (`N_t = int(T_LUNAR / DT_STEP) + 1` in `run_with`). The test suite samples more coarsely (2-hour steps) to keep continuous-integration runs fast.

A handy units gotcha. Watch out for the number 2551: it looks like a timestep count but is really the lunation *length* in thousands of seconds (2 551 443 s). The number of hourly steps is **709**.

4.3 Crank–Nicolson: a stable, accurate way to take a step

There are simple ways to step a heat equation forward, but the naïve ones either blow up (become numerically unstable) unless the timestep is microscopically small, or they are inaccurate. **Crank–Nicolson** is the standard professional choice. Intuition:

- An **explicit** step computes the future purely from the present. Cheap, but unstable for stiff problems like heat (it can oscillate and explode).
- An **implicit** step computes the future from the future (you solve a small system of equations). Stable, but slightly less accurate.
- **Crank–Nicolson averages the two** (“half now, half later,” the 0.5 factors you’ll see in the code). The result is both **stable** and **second-order accurate** in time. It is coded in `_step` in `lunar/solver.py` (look for the `0.5 * alpha_...` terms — that 0.5 is the Crank–Nicolson averaging).

You don't need to derive it. Just know: Crank–Nicolson is the recipe that lets us take comfortable one-hour steps without the simulation exploding.

4.4 The tridiagonal (Thomas) solver: the fast inner engine

An implicit / Crank–Nicolson step requires solving a system of linear equations: “find all the new-layer temperatures at once such that they are mutually consistent.” Normally solving N equations in N unknowns is slow (cost grows like N^3). But here each layer only talks to its **immediate neighbours above and below** — so the system is **tridiagonal** (nonzero only on the diagonal and the two adjacent diagonals).

For tridiagonal systems there is a famous shortcut, the **Thomas algorithm**, that solves them in a single sweep down and a single sweep back up — cost grows only like N (linear), which is as fast as it gets. It is implemented in `_thomas` in `lunar/solver.py`:

Listing 4. The Thomas tridiagonal solver, `lunar/solver.py`.

```
1 @njit(cache=True)
2 def _thomas(a, b, c, d):
3     """Thomas algorithm for an (a, b, c) tridiagonal system A x = d."""
4     n = b.size
5     cp = np.empty(n); dp = np.empty(n); x = np.empty(n)
6     cp[0] = c[0] / b[0]
7     dp[0] = d[0] / b[0]
8     for i in range(1, n):          # forward sweep
9         m = b[i] - a[i] * cp[i - 1]
10        cp[i] = c[i] / m if i < n - 1 else 0.0
11        dp[i] = (d[i] - a[i] * dp[i - 1]) / m
12    x[n - 1] = dp[n - 1]
13    for i in range(n - 2, -1, -1):  # back substitution
14        x[i] = dp[i] - cp[i] * x[i + 1]
15    return x
```

The `@njit` decorator is **Numba**, a tool that compiles this Python down to machine code so it runs $\sim 10\text{--}100\times$ faster (this inner loop runs millions of times). If Numba isn't installed, the code falls back to plain Python and still works — see the `try/except` at the top of `lunar/solver.py`. (`docs/` contains `NUMBA_VS_CPP_JUSTIFICATION.md` explaining why Numba was chosen over writing C++.)

One more detail: conductivity between two adjacent layers uses the **harmonic mean** (`_face_harmonic_mean`), not the simple average. For heat flowing *through* two materials in series (like the deep, conductive layer feeding the fluffy, insulating layer above), the harmonic mean is the physically correct combination — the poor conductor dominates, just as the narrowest pipe dominates a series of pipes.

4.5 Newton's method for the non-linear surface

Remember the surface radiates as T_s^4 (subsection 3.7). That fourth power makes the surface energy-balance equation **non-linear** — you can't just solve for T_s algebraically. Instead we use **Newton's method**, the classic “guess, check the error, slide toward the answer, repeat” root-finder:

1. Guess a surface temperature T_s .
2. Compute the **residual** $R(T_s)$ — how badly the energy balance fails (absorbed – radiated – conducted; `surface_energy_balance_residual`).

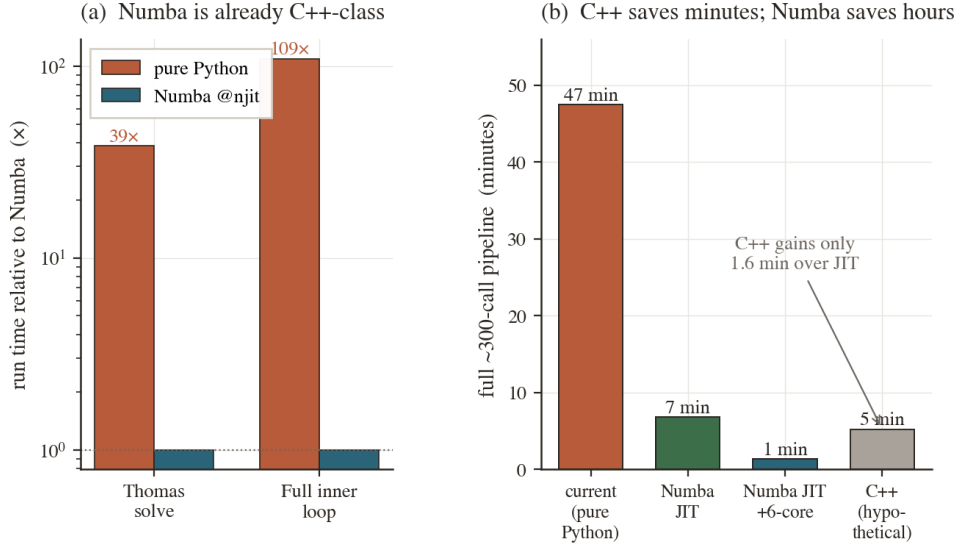


Figure 9. Why the `@njit` compiler matters: a benchmark of the solver’s inner loop. **What to notice:** compiling the hot Python loop with Numba closes almost all of the gap to hand-written C++, while keeping the code in one readable language. **Why it matters:** the retrieval runs the solver hundreds of times, so this speed-up is what makes the whole study practical on a laptop.

3. Use the slope dR/dT_s to jump to a better guess.
4. Repeat until R is ~ 0 (a few iterations).

This is `_solve_surface_newton` in `lunar/solver.py`. The code notes the derivative is always negative, which guarantees Newton converges from any positive starting temperature. So at **every hourly step**, the solver does a tiny Newton solve to pin the surface temperature, then a Thomas sweep to update the whole column. That’s the complete inner loop.

5 The Anchor Point Method (the centerpiece)

This is the **centerpiece** of the project — the clever idea that makes the whole thing fast *and* trustworthy. It lives in `lunar/equilibrium.py`. Read that file’s docstring; it’s excellent. This section builds on it.

5.1 The problem: the deep Moon settles agonizingly slowly

We want the **periodic steady state**: the repeating day-after-day temperature pattern the soil settles into after a long time, when every lunar day looks like the last. To get there the obvious approach is “spin-up” — just press play and simulate many lunar days until things stop changing.

The catch: the **deep** column relaxes on the *diffusion timescale of the whole 5-metre column*, which is roughly **1000 lunar days**. Simulating 1000 lunations per run, for the ~ 300 runs the retrieval needs, is hopeless.

Worse, if you stop early (say after 30 lunations), the deep temperature still **secretly remembers your starting guess**. And the deep temperature is exactly what the Apollo sensors measure. So your “measurement” of K_d would be contaminated by an arbitrary guess you typed in. This was a real, critical bug caught in the project’s own audit — **flag F1** in `docs/FLAG_REPORT.md`. The original code used hard-coded starting temperatures (250 K for A15,

255 K for A17), and a ± 5 K change in that guess swung the retrieved K_d^* by a factor of ~ 6 — *larger than the very inter-site contrast the paper reports!* That had to be fixed before any result could be believed.

5.2 The key insight: two clocks

The fix comes from realizing the soil has **two very different “clocks”**:

- **The shallow skin (top $\sim$ half metre) is FAST.** It settles into its repeating day/night rhythm within a handful of lunar days, because it is thin and driven hard by the surface.
- **The deep column is SLOW** to settle by brute force — *but* in steady state it must obey one dead-simple rule: the same steady interior heat Q_b flows through every layer. That single rule, $d\langle T \rangle/dz = Q_b/K$, **fixes the entire deep temperature shape instantly** — no waiting required. ($\langle T \rangle$ means the cycle-averaged temperature; the angle brackets denote “average over one day.”)

So instead of waiting 1000 lunations for the deep column, we **calculate** it from the steady-flux rule, and only ever simulate the fast skin.

5.2.1 Where $d\langle T \rangle/dz = Q_b/K$ comes from

Same physics as [subsection 3.7](#) (geothermal gradient), now as the anchor shortcut:

1. Time-average the heat equation over one lunation.
2. In periodic steady state: $\partial\langle T \rangle/\partial t = 0$.
3. Equation collapses to $\partial[K \partial\langle T \rangle/\partial z]/\partial z = 0$.
4. So $K d\langle T \rangle/dz = Q_b$ everywhere below the skin.
5. Integrate downward from the anchor: `_reconstruct_subskin` in `lunar/equilibrium.py`.

5.2.2 The rectified flux u_{rect}

Inside the skin, $\langle K \partial T/\partial z \rangle \neq K(\langle T \rangle) d\langle T \rangle/dz$. The extra **rectification** term u_{rect} is measured from the resolved cycle (`_rectified_flux`) and subtracted: $d\langle T \rangle/dz = (Q_b - u_{\text{rect}})/K$. Negligible below 1 m; a few percent near 0.3 m–0.5 m — why the anchor sits at 0.55 m.

5.3 The method: alternate “settle the skin” and “rebuild the deep”

The algorithm (the “outer iteration”) alternates two cheap steps until they agree:

1. **Settle the skin.** Run the full solver for just a few lunar days (`n_inner` = 12) starting from the current profile. This locks the fast skin into rhythm against the current deep column.
2. **Rebuild the deep.** Read the cycle-mean temperature at one trusted **anchor depth** just below the skin (`z_anchor` = 0.55 m), then integrate the steady-flux rule $d\langle T \rangle/dz = (Q_b - u_{\text{rect}})/K$ *downward* from there to reconstruct the entire deep profile exactly.

Repeat steps 1–2 a few times (typically 3–5). The result “locks on” to the true periodic steady state. The actual loop is `solve_periodic_equilibrium` in `lunar/equilibrium.py`; the deep rebuild is `_reconstruct_subskin`.

Two refinements worth knowing:

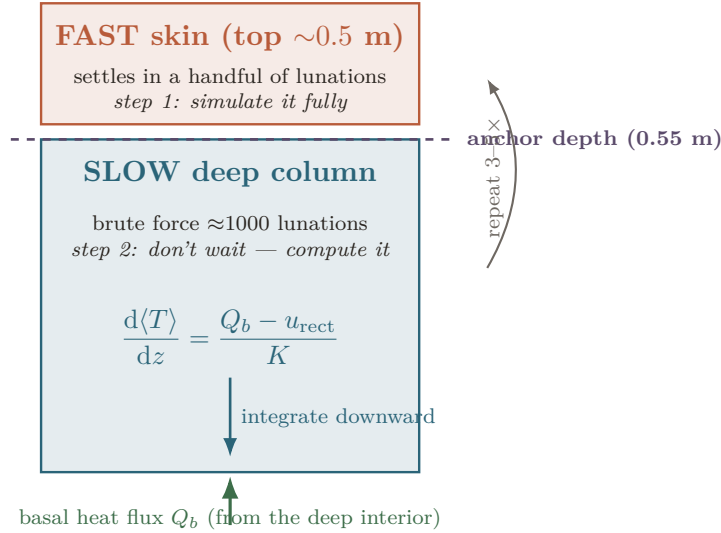


Figure 10. The Anchor Point Method as “two clocks.” **What to notice:** the fast skin is simulated in full (step 1); the slow deep column is *reconstructed* from the steady-flux rule (step 2) rather than waited out, anchored to the cycle-mean temperature read at 0.55 m. The two steps are repeated a few times until they agree. **Why it matters:** this turns an impossible ~ 1000 -lunation wait into a handful of cheap iterations, while removing the starting-guess bias.

- u_{rect} — **the rectified flux.** Inside the skin, the daily wobble plus the fact that conductivity depends on temperature creates a small extra *day–night-averaged* heat term (an “eddy” or “rectification” term). The code measures it exactly from the resolved cycle (`_rectified_flux`) and subtracts it, so the deep rebuild stays accurate. It is negligible below ~ 1 m but matters a few percent right at the anchor — which is precisely why the anchor is placed at 0.55 m, *below* the worst of it.
- **Two-stage anchor schedule.** The code first anchors high (0.25 m, relaxes fast) to cheaply kill most of the starting-guess error, then drops to 0.55 m for the accurate finish. See the `stages` tuple in `solve_periodic_equilibrium`.

5.4 Why it’s both fast *and* unbiased

- **Fast:** it only ever simulates the quick skin (a few lunations per outer step, ~ 36 –60 lunations total) instead of ~ 1000 . Comparable cost to the old broken spin-up, but correct.
- **Unbiased (this is the crucial part):** the final answer **no longer depends on the starting guess**. The skin is forced periodic by step 1; the deep is forced to satisfy steady flux by step 2. The only fixed point of that two-step map is the *true* equilibrium. The starting guess just seeds the first iterate and gets erased.

This is **certified, not assumed**. The regression test `tests/test_equilibrium.py::test_guess_independence` literally runs the solver from two different starting guesses (242 K and 258 K) and asserts the converged profiles agree:

Listing 5. The guess-independence regression test.

```
1 def test_guess_independence():
2     """Converged profiles from 242 K and 258 K guesses must agree."""
3     grid, t, insol, k_func, cp_func = _setup()
4     profiles = []
5     for guess in (242.0, 258.0):
6         eq = solve_periodic_equilibrium(
7             grid=grid, t=t, insolation=insol, albedo=0.131,
```

```

8         emissivity=0.95, Q_b=0.021, K_func=k_func, cp_func=cp_func,
9         T_guess=guess)
10     assert eq.converged
11     profiles.append(eq.T_mean)
12     assert np.max(np.abs(profiles[0] - profiles[1])) < 0.1

```

In the full production setting the guess-dependence is ≤ 0.023 K (vs the old 1.6–4.5 K). That is what “F1 RESOLVED” at the top of `docs/FLAG_REPORT.md` means, and it’s why the retrieved K_d^* values can be trusted.

5.5 “Flux closure” — how the code proves it worked

How do you know the deep profile really satisfies steady heat flow? You check whether the cycle-mean conductive flux equals Q_b at every depth. The maximum relative deviation is the **flux closure** number (`flux_closure` in `EquilibriumResult`). The retrieval warns if closure is worse than 3%. It is deliberately measured **below the diurnal skin** (where the rule actually applies and where every Apollo sensor sits), not at the noisy surface. The test `test_flux_closure` checks this. The `make_equilibrium_certification.py` figure script visualizes the whole certification.

6 The K_d retrieval and uncertainty analysis

Now we connect the simulator to the real Apollo data and extract K_d . The driver is `scripts/pipeline/retrieve_kd`.

6.1 The data: turning raw HFE records into “equilibrium temperatures”

The bundled Apollo data lives in `data/apollo/depth/` as `.tab` text files (one per mission per probe, e.g. `a15p1_depth.tab`). Each row is a timestamp, a temperature, a sensor name, and the sensor’s depth in cm. The loader is `load_apollo_hfe_depth` (`lunar/validation.py`).

But the raw record drifts for years after the probes were emplaced (the drilling disturbed the soil, which slowly re-equilibrated). We only want the final **settled** temperature of each sensor. `find_stable_window` (`lunar/apollo_helpers.py`) scans the late part of each sensor’s record and picks the earliest window where the temperature has gone **flat** (trend slope below 0.08 K/year), then averages it. That average is the sensor’s **equilibrium temperature** T_{eq} . `extract_sensor_stability` does this for every sensor and returns depths + T_{eq} values.

Crucially, only sensors **deeper than** `MIN_DEPTH_CM = 80 cm` are used for the fit (the `deep_mask`). Shallower sensors are contaminated by the borestem (the hardware sticking up) and by the daily skin, so they are excluded *a priori*.

6.2 The retrieval: sweep K_d , minimize RMSE

The core idea is a **sweep** (`run_kd_sweep_extended`):

1. Pick a grid of candidate K_d values (e.g. 28 values from 1 to 15 mW for A15; see `KD_GRIDS` in `config.py`).
2. For each candidate K_d , run the equilibrium solver (`run_with`, which calls `solve_periodic_equilibrium`) to get a predicted temperature-vs-depth profile.
3. Read the model temperature at each real sensor depth and subtract the measured T_{eq} . That difference is the **residual**.
4. Combine the residuals into one number, the **RMSE**.

RMSE = Root Mean Square Error. It is the standard “how far off, on average” score: square every residual (so positives and negatives don’t cancel and big misses count extra), average them, take the square root. Smaller = better fit. Units are kelvin here:

$$\text{RMSE}(K_d) = \sqrt{\frac{1}{N} \sum_{i=1}^N [T_{\text{model}}(z_i; K_d) - T_{\text{eq},i}]^2}.$$

The best-fit K_d^* is the value that **minimizes** RMSE. The code even fits a little parabola through the three points around the minimum to find the bottom precisely (`kd_star_from_residuals`). That’s the whole retrieval: *slide the one knob K_d until the model temperatures best match the measured ones.*

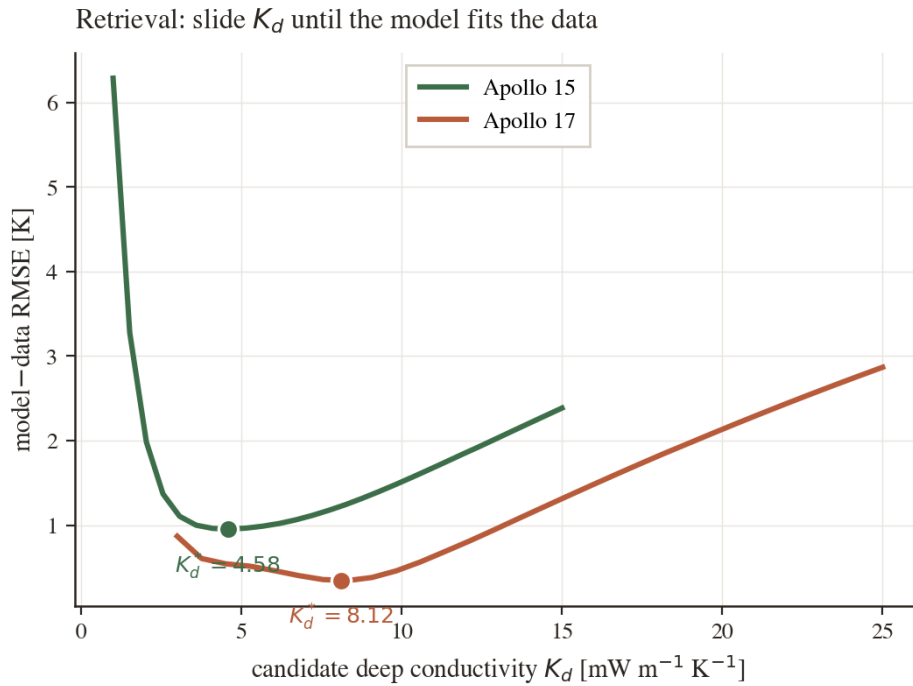


Figure 11. The retrieval curve, computed from the committed results. For each candidate deep conductivity K_d we run the simulator and score how far its temperatures land from the Apollo measurements (RMSE). **What to notice:** each site’s curve is a clean valley; its lowest point is the retrieved K_d^* (filled marker) — $4.58 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ for Apollo 15 and $8.12 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ for Apollo 17. **Why it matters:** the two valleys sit at clearly different places, which is the visual heart of the “the sites differ” result.

Result: A15 lands at $K_d^* \approx 4.58$, A17 at $\approx 8.12 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$, and switching to per-site values roughly **halves** the Apollo-17 mismatch versus the old global $K_d = 3.4$.

6.3 Uncertainty: the bootstrap

A single best-fit number isn’t science without an error bar. How sure are we? The biggest uncertainty is that the **sensor depths themselves** are only known to about $\pm 2.5 \text{ cm}$ (Nagihara 2018). The repo propagates this with a **bootstrap** (`bootstrap_kd_with_depth_uncertainty`):

- Repeat 1500 times: randomly **resample** the sensors (draw with replacement) *and jitter* each depth by a random $\pm 2.5 \text{ cm}$ wiggle, then re-find K_d^* .
- You end up with 1500 slightly different K_d^* values — a whole distribution.

- The middle 95% of that distribution is the **95% confidence interval** (the $+1.33/-0.28$ style error bars).

A bootstrap is just “shake the inputs within their uncertainty, many times, and watch how much the answer wobbles.” The wobble *is* the error bar. (The bootstrap reuses cached solver outputs, so it is pure fast statistics — no new physics runs. It is $\sim 65\%$ of the pipeline’s runtime purely because of the 1500 repeats.)

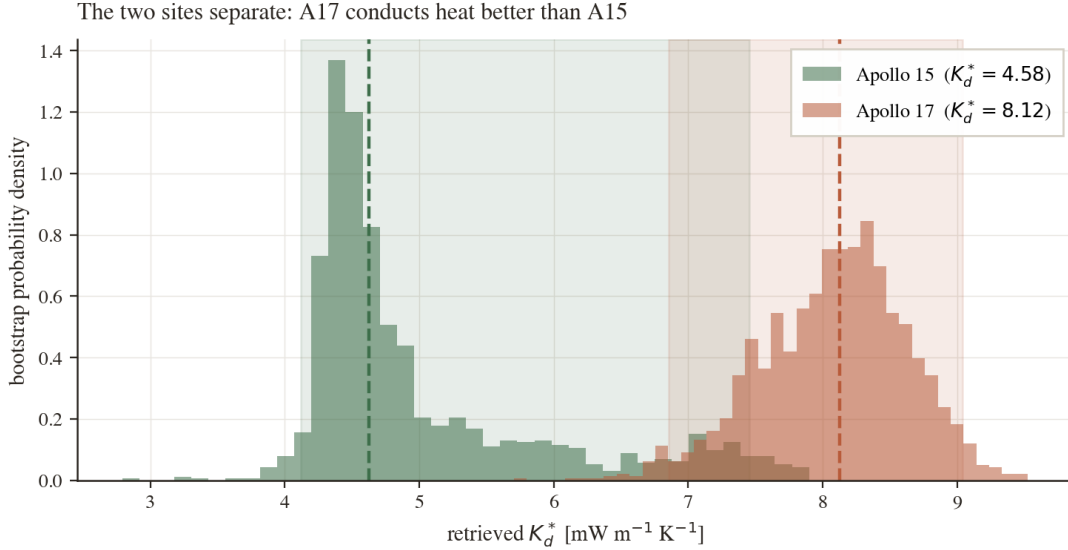


Figure 12. The bootstrap distributions of K_d^* for both sites. Each histogram is the spread of answers you get when the sensor depths are jittered within their ± 2.5 cm uncertainty and the sensors are resampled, 1500 times. Dashed lines mark the medians; shaded bands mark the 95% confidence intervals. **What to notice:** the two clouds barely overlap. **Why it matters:** that separation is the quantitative statement that Apollo 17 conducts heat better than Apollo 15 — the contrast’s confidence interval excludes zero ($p \approx 0.01$).

6.4 Inter-site contrast and the K_d – Q_b degeneracy

The headline scientific claim is the **contrast**: $A17 - A15 \approx 3.3 \text{ mW} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$. Because we have a bootstrap distribution for *each* site, we can subtract them sample-by-sample to get a distribution for the **difference**, and check whether it excludes zero. It does, at the published heat fluxes: 95% CI $\approx [0.4, 4.6]$, $p \approx 0.01$ (contrast_bootstrap in the results JSON).

The big caveat the paper is honest about: deep temperature depends on the **ratio** Q_b/K_d , so K_d and the basal flux Q_b are partly **degenerate** — if you assumed a different Q_b , you would retrieve a different K_d . That is why every result is stated “*conditional on the published basal heat fluxes.*” Propagating the full uncertainty in Q_b dilutes the significance (the ordering probability drops to $\sim 83\%$).

The honest summary: the sites differ in the combination $T(z)$ they produce; attributing that purely to conductivity rather than basal flux rests on the published Q_b values.

Several auxiliary scripts probe robustness (held-out tests, model selection, borestem cut, an independent Bayesian/MCMC cross-check) — see [section 7](#).

6.5 The optional bedrock toggle (for future ice work)

Recently added: an **optional bedrock layer** (`with_bedrock` in `lunar/properties.py`, configured by `BEDROCK` in `lunar/config.py`). The standard model treats the Moon as fluffy regolith all the way down. In reality, below ~ 10 m there is solid rock, which conducts far better. `with_bedrock` wraps any conductivity model so that below `z_bedrock` (10 m) the conductivity ramps smoothly (via a `tanh`) up to a rock value (`K_rock` = 2.0).

It is **OFF by default** and **does not change the published retrieval** — every Apollo sensor is at ≤ 2.4 m, far above the 10 m transition, where enabling bedrock shifts temperatures by only ~ 0.05 K. The tests `tests/test_bedrock.py` lock in both guarantees (off by default; sensor-depth temperatures unchanged). It exists for **future deep-profile / ice-stability studies**, where the temperature far below the regolith *does* matter.

7 Code tour: every file, in plain terms

A file-by-file map. Two layers: the `lunar/` **package** is the reusable engine (all the physics and config); the `scripts/` are thin command-line drivers that call the engine and write results/figures. There is exactly one definition of everything — config never gets copy-pasted.

7.1 The `lunar/` package (the engine)

File	What it does, in plain terms
<code>__init__.py</code>	Marks the folder as a package; sets the version. Nothing to study.
<code>constants.py</code>	The physical numbers : solar constant, Stefan–Boltzmann, regolith $K_s/K_d/H/\chi$, density / specific-heat coefficients, basal fluxes. Every value is cited to a paper. Project rule: no unsourced numbers.
<code>config.py</code>	The settings menu / single source of truth : the two-site table (SITES), the depth grid (GRID), the Hayne bundle (HAYNE), equilibrium-solver knobs, the K_d sweep grids, the bedrock toggle. Change “what the experiment is” here.
<code>grid.py</code>	Builds the geometric depth grid (<code>make_geometric_grid</code> → a <code>DepthGrid</code> of layer faces, centres, thicknesses). subsection 4.1 .
<code>properties.py</code>	The soil property formulas : conductivity (<code>conductivity_hayne</code> , <code>conductivity_martinez</code> , <code>conductivity_icy</code>), density (<code>density_hayne</code>), specific heat (<code>specific_heat</code>), and the optional <code>with_bedrock</code> wrapper. subsection 3.4–3.5 .
<code>solver.py</code>	The heat-equation engine : one Crank–Nicolson step (<code>_step</code>), the tridiagonal <code>_thomas</code> solver, harmonic-mean faces, the Newton surface solve (<code>_solve_surface_newton</code>), the driver <code>solve_pixel</code> , and the analytical wave used for testing. section 4 .

File	What it does, in plain terms
equilibrium.py	The Anchor Point Method : <code>solve_periodic_equilibrium</code> plus helpers (<code>_rectified_flux</code> , <code>_reconstruct_subskin</code> , <code>_diurnal_skin_index</code> , <code>_mean_flux_closure</code>). Returns an <code>EquilibriumResult</code> . section 5 . Read this docstring .
validation.py	Loads the bundled Apollo HFE .tab data into numpy arrays (<code>load_apollo_hfe_depth</code>).
apollo_helpers.py	Turns raw HFE records into per-sensor equilibrium temperatures : <code>find_stable_window</code> (detect the settled tail) and <code>extract_sensor_stability</code> . subsection 6.1 .
ephem.py	A correct SPICE/DE440 solar-position helper (sun elevation $\rightarrow$ insolation). <i>Note</i> : the K_d pipeline uses the simpler idealized cosine forcing, not this.
diviner.py	Helpers for Diviner orbital surface-temperature data, used for the surface-temperature “closure” cross-check.
plotting/style.py	Shared figure styling (JGR:Planets sizes, colour palette, layout helpers). Importing it gives every figure a consistent look.
_bootstrap.py	Small environment bootstrap so standalone scripts can find the package and the data. Plumbing, not physics.

7.2 scripts/pipeline/ (compute results $\rightarrow$ write output/*.json)

File	What it produces
retrieve_kd.py	The main event. Loads HFE data, runs the K_d sweep + bootstrap + joint (K_d, H) grid + held-out tests, writes <code>output/kd_retrieval_results.json</code> and the bootstrap/robustness/holdout figures. Start here.
compute_headline_rmse.py	The headline RMSE-vs- K_d numbers (a second grid as a cross-check).
compute_borestem_sensitivity.py	How K_d^* changes if you move the shallow-sensor cutoff (the borestem-contamination test).
compute_stability_threshold_sensitivity.py	How results depend on the “flatness” threshold used to detect equilibrium windows.
compute_surface_bias_test.py	Tests for systematic surface-temperature bias.
compute_uniform_kd_sensitivity.py	What you’d get with a single uniform K_d (the “old way”) for comparison.
compute_fixed_input_sensitivities.py	How K_d^* responds to fixed inputs (albedo, H , etc.) — feeds the error budget.
compute_model_selection.py	Compares competing conductivity models (AICc model selection).
compute_error_budget.py	Assembles the full systematic error budget (paper Table 4).
bayesian_crosscheck.py	An independent MCMC/Bayesian retrieval as a cross-check on the bootstrap.
compute_diviner_closure.py	Checks the model’s surface temperature against orbital Diviner data.

7.3 scripts/figures/ and scripts/analysis/

- `scripts/make_all_figures.py` — runs **every** figure generator in order.
- `make_letter_figures.py`, `make_results_figures.py`, `make_intro_figures.py` — the paper’s figures.
- `make_equilibrium_certification.py`, `make_equilibrium_demo.py`, and the animation scripts — visualize the Anchor Point Method (great for understanding [section 5](#)).
- `make_primer_figures.py`, `make_book_figures.py`, `make_codeguide_figures.py` — teaching figures for the manuscript/guidebook.
- `scripts/analysis/phase2_depth_convergence.py` — the standalone “what happens deep down?” diagnostic ([subsection 3.6](#)). A great first script to *run and read* (see [Figure 13](#)).
- `scripts/fetch_diviner.py` — one-time download of Diviner data (~310 MB).
- `scripts/run_all_timed.py` — runs the whole pipeline with timing.

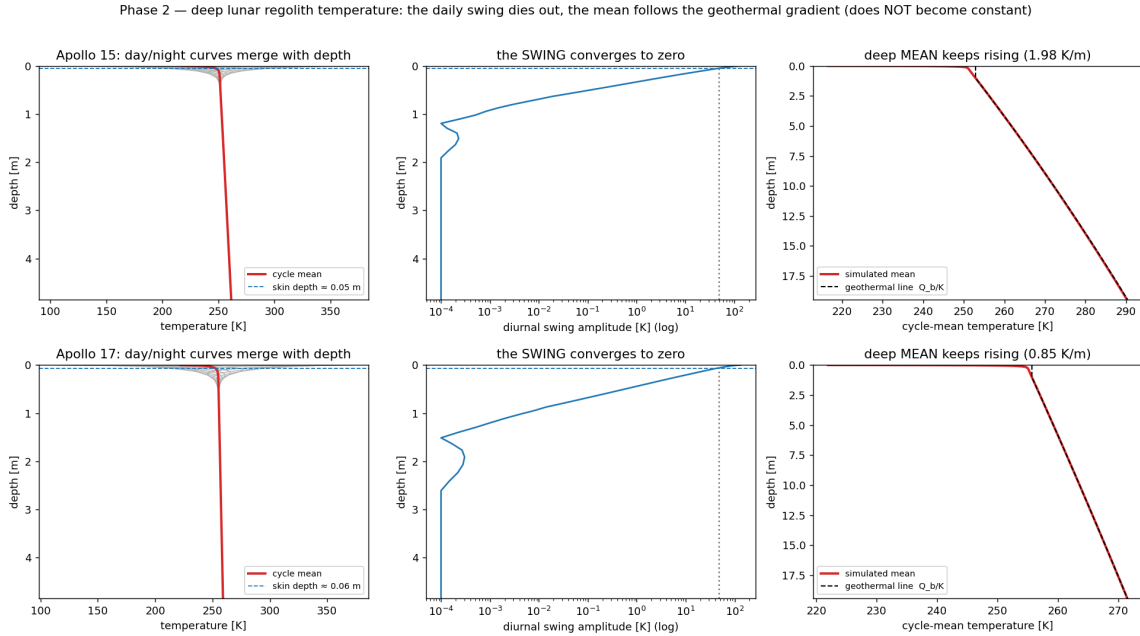


Figure 13. The committed `phase2` diagnostic, reproduced here for reference. **What to notice:** the left panels show every-few-hours temperature curves merging onto a single line within ~1 m; the middle panels show the swing damping toward zero; the right panels show the cycle-mean continuing to rise along the geothermal line Q_b/K . **Why it matters:** it is the same physics as [Figure 2–3](#), shown end to end on one sheet — a superb on-ramp to the project.

7.4 The end-to-end data flow

[Figure 14](#) traces a single run from cited constants all the way to the paper. Read it top-to-bottom: cited constants feed the config; the config plus the property/grid modules feed the solver; the solver wrapped in the Anchor Point Method produces a steady temperature profile; the retrieval slides K_d to match that profile to the processed Apollo data; the results become JSON, figures, and ultimately the paper.

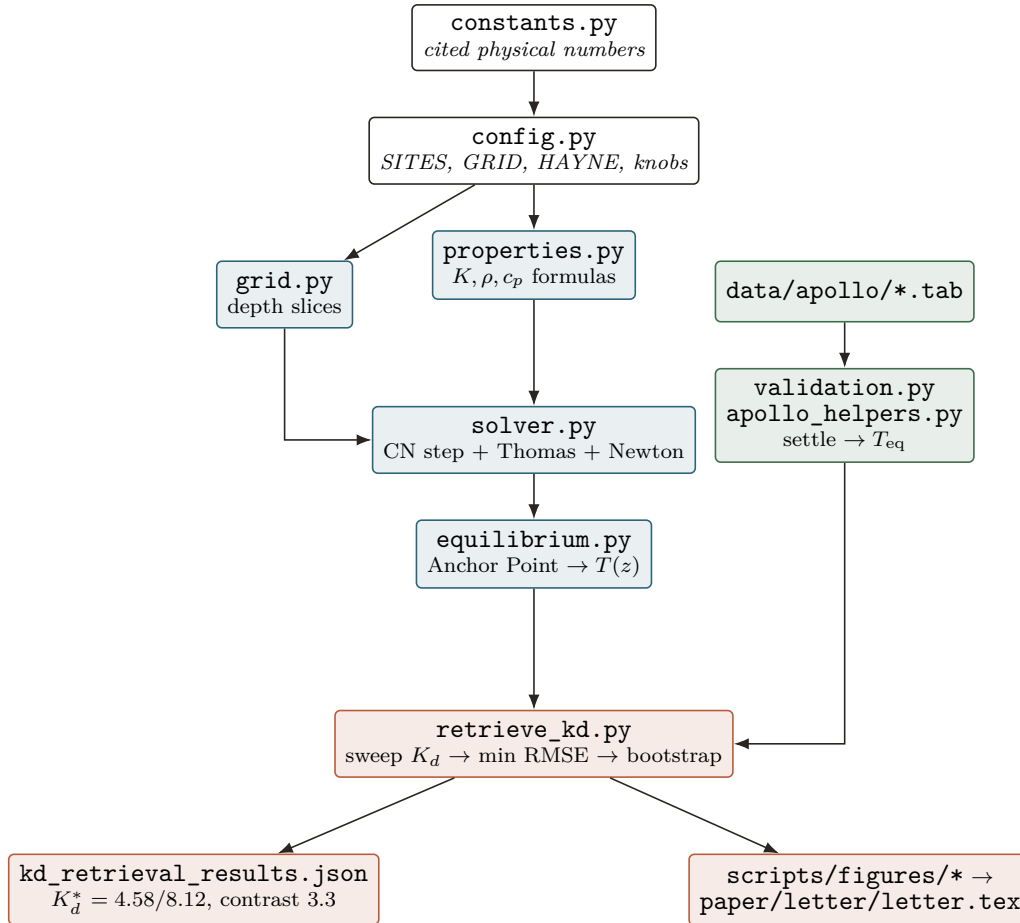


Figure 14. The end-to-end pipeline. **Teal** boxes are the physics engine, **green** boxes are the data path, and **coral** boxes are the outputs. **What to notice:** everything funnels through one retrieval driver that compares simulated profiles to processed Apollo temperatures. **Why it matters:** there is exactly one path from a cited constant to a published number — nothing is hand-edited along the way.

8 How to run it yourself

These commands are cross-checked against `docs/REPRODUCING.md` and the `Makefile`. Run them from the repository root.

8.1 One-time setup

Listing 6. Create an isolated environment and install the package.

```

1 # 1. Create an isolated Python environment (so you don't clutter your system)
2 python3 -m venv .venv
3 source .venv/bin/activate           # Windows: .venv\Scripts\activate
4
5 # 2. Install the package + dev tools (pytest, jupyterlab, matplotlib, ...)
6 make install                       # same as: pip install -e ".[dev]"

```

A **virtual environment** (`venv`) is just a private sandbox of Python packages for this project. `pip install -e .` installs the `lunar` package in “editable” mode, meaning Python sees your local `lunar/` folder directly — edit a file and the change takes effect immediately.

8.2 Run the tests first (always do this)

```
1 python3 -m pytest -q
```

This runs the suite in `tests/` (grid, properties, solver, equilibrium, bedrock, ephem, constants). They finish in under a minute and should **all pass**. Reading a test is often the clearest example of how a function is meant to be used.

8.3 Verify the committed headline numbers (no computation needed)

The canonical result ships in the repo, so you can check it instantly:

Listing 7. Verify the committed headline K_d^* values.

```
1 import json, math
2 r = json.load(open('output/kd_retrieval_results.json'))
3 print('A15 K_d* =', r['A15']['kd_star'] * 1e3, 'mW/m/K')
4 print('A17 K_d* =', r['A17']['kd_star'] * 1e3, 'mW/m/K')
5 assert math.isclose(r['A15']['kd_star']*1e3, 4.58, abs_tol=0.01)
6 assert math.isclose(r['A17']['kd_star']*1e3, 8.12, abs_tol=0.01)
7 print('Headline values verified.')
```

8.4 Reproduce the science (recompute from scratch)

Listing 8. The one-word Makefile targets.

```
1 make help          # list every one-word command
2 make retrieve      # the core K_d retrieval + bootstrap (~5 min)
3                   # -> writes output/kd_retrieval_results.json
4 make aux           # all auxiliary sweeps, model selection, error budget, MCMC
5 make figures       # regenerate every figure
6 make paper         # compile the PDFs (needs a LaTeX install)
7 make all           # everything end-to-end
```

`make retrieve` is the one to run first — it regenerates the headline JSON. The slow auxiliary sweeps (`make aux`) are rarely needed because their results are already committed.

8.5 Run a single illuminating script

To *see* the deep-temperature physics (subsection 3.6) with your own eyes:

```
1 python3 scripts/analysis/phase2_depth_convergence.py
```

It prints the skin depth, the deep gradient, and confirms the mean temperature keeps rising along Q_b/K , then saves `output/figures/fig_phase2_depth_convergence.png` (Figure 13). Open that image. This is one of the best on-ramps to understanding the project.

Regenerate the five teaching figures in this very handbook. From the repository root:

```
1 export MPLCONFIGDIR=/tmp/mpl
2 python3 docs/teaching/make_teaching_figures.py
```

This re-runs the certified equilibrium solver for both sites and re-reads the committed results JSON, writing fresh PNG+PDF assets into `docs/teaching/figures/`. Then rebuild this PDF with `latexmk -pdf beginners_guide.tex`.

8.6 The notebooks (guided, in order)

```
1 jupyter lab notebooks/
```

Run them in order: 00_setup (checks), 01_methods (figures + parameter table), 02_retrieval (the K_d retrieval + bootstrap), 03_results, 04_discussion, 05_animations. There's also `equilibrium_demo.ipynb` a standalone demo that a brute-force 1000-lunation spin-up converges onto the fast Anchor Point solver — a great way to *believe* [section 5](#).

8.7 Where outputs land

- `output/*.json` — numerical results (the canonical `kd_retrieval_results.json` and the auxiliary JSONs).
- `output/figures/*.png / *.pdf` — diagnostic and appendix figures.
- `paper/letter/figures/*.pdf` — the paper's figures.
- `paper/letter/letter.pdf` — the compiled manuscript.

9 Glossary

Plain one-liners. Physics, then numerical, then code/project terms.

Physics

Regolith

The loose broken rock and dust covering the Moon; the “soil.”

Temperature (K, kelvin)

How hot something is; a scale starting at absolute zero ($0\text{ K} = -273.15^\circ\text{C}$).

Heat flux

Rate of heat energy flow through a unit area (W/m^2).

Conduction

Heat moving by direct contact, atom to atom.

Thermal conductivity K

How easily a material conducts heat ($\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$); high = metal, low = lunar dust.

K_d

The *deep* regolith conductivity; the quantity this project retrieves. K_d^* is the best-fit value.

K_s

The *surface* regolith conductivity (very low).

H (scale height)

Sets how fast conductivity / density transition from surface to deep values ($\sim 6\text{ cm}$).

χ (chi, radiative coefficient)

Strength of the temperature-dependent radiative boost to conductivity (the T^3 term).

Density ρ (rho)

Mass per unit volume (kg/m^3).

Specific heat c_p

Energy to warm 1 kg by 1 K ($\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$).

Thermal diffusivity α

$K/(\rho c_p)$; how fast a temperature change spreads through a material.

Heat equation

The master PDE: $\rho c_p \partial T / \partial t = \partial / \partial z (K \partial T / \partial z)$. Derived step by step in [subsection 3.3](#) from energy conservation in a thin soil slice plus Fourier’s law.

Heat flux q

Thermal energy crossing a surface per second per m^2 (W/m^2); $q = -K \partial T / \partial z$.

Boundary condition

A rule at the domain edge that closes the PDE — here, radiative balance at the surface and fixed geothermal flux Q_b at depth.

Skin depth δ

Depth over which a daily temperature wave fades to $\sim 1/3$; a few cm on the Moon.

Insolation S

Incoming solar power per area at the surface (W/m^2).

Albedo A

Fraction of sunlight reflected (≈ 0.13 here).

Emissivity ε

How efficiently a surface radiates heat (≈ 0.95).

Stefan–Boltzmann law

Radiated power $\propto T^4$; the σ is its constant.

Geothermal / basal heat flux Q_b

Steady heat seeping up from the Moon’s interior ($\sim 15\text{--}21 \text{ mW}/\text{m}^2$).

Periodic steady state / equilibrium

The repeating day-after-day temperature pattern reached after long settling.

Diurnal

Daily (here, one lunar day ≈ 29.5 Earth days, a “lunation”).

Numerical**Discretize**

Replace continuous space/time with finite slices/steps so a computer can solve it.

Grid

The set of depth slices; here *geometric* (each slice $\sim 8\%$ thicker than the one above).

Timestep Δt

The time jump per update (1 hour here).

Crank–Nicolson

A stable, accurate “half-now, half-later” time-stepping scheme.

Tridiagonal system

A linear system where each unknown only couples to its two neighbours.

Thomas algorithm

The fast (linear-cost) solver for tridiagonal systems.

Harmonic mean

The correct way to average conductivity between two layers in series.

Newton’s method

Iterative root-finder used for the non-linear surface temperature.

Residual

How badly an equation fails for a trial value (model – data, or the energy-balance mismatch).

RMSE

Root-mean-square error; the average size of the model-vs-data mismatch (K).

Bootstrap

Estimate uncertainty by resampling/jittering inputs many times and watching the answer wobble.

Confidence interval (CI)

The range that contains the answer with stated probability (e.g. 95%).

Degeneracy

When two parameters trade off so only their combination is constrained (here K_d and Q_b).

Code / project**Anchor Point Method**

This project’s fast, unbiased equilibrium solver ([section 5](#)); the “two clocks” idea.

Flux closure

Diagnostic checking the mean conductive flux equals Q_b at depth (proof the solver converged).

 u_{rect} (**rectified flux**)

Small day-night-averaged eddy heat term inside the skin, measured and subtracted.

Spin-up

Running many cycles to settle a simulation (the slow approach the Anchor Point Method replaces).

F1

The audit flag (in `docs/FLAG_REPORT.md`) for the old starting-guess-bias bug that the Anchor Point Method fixed.

HFE

Apollo Heat-Flow Experiment (the buried thermometers).

Numba / @njit

A just-in-time compiler that speeds up the hot inner loops $\sim 10\text{--}100\times$.

venv

An isolated Python environment for this project.

pytest

The tool that runs the test suite.

10 A learning roadmap: what to study next

A suggested order to go from “I read this guide” to “I can change the code.”

Day 1 — orient and run

1. Re-read [section 1](#) and [section 3.1–3.3](#) until “heat $\neq$ temperature” and the heat equation feel natural.
2. Do the setup ([section 8.1](#)), run the tests ([section 8.2](#)), verify the headline numbers ([section 8.3](#)).

3. Run `scripts/analysis/phase2_depth_convergence.py` (section 8.5) and stare at the output figure while re-reading subsection 3.6. This makes skin depth and the geothermal gradient *concrete*.

Day 2 — read the engine

4. Read `lunar/constants.py` then `lunar/config.py` end to end — they are short and define “the experiment.”
5. Read `lunar/properties.py` (`conductivity_hayne`, `density_hayne`, `specific_heat`) alongside subsection 3.4–3.5.
6. Read `lunar/grid.py` alongside subsection 4.1.
7. Skim `lunar/solver.py`: focus on the *docstring*, `surface_energy_balance_residual`, and `_thomas`. Don’t try to absorb every index in `_step` yet.

Day 3 — the centerpiece

8. Read the full docstring of `lunar/equilibrium.py` (it is a teaching text in itself), then `solve_periodic_equilibrium`, with section 5 open beside it.
9. Read `tests/test_equilibrium.py` — the tests *are* the precise statement of what “correct” means here.
10. Optional: run `equilibrium_demo.ipynb` to watch brute-force converge onto the fast method.

Day 4 — the science

11. Read `scripts/pipeline/retrieve_kd.py` top to bottom with section 6 open. Follow one K_d value through `run_with` → `run_kd_sweep_extended` → `kd_star_from_residuals`.
12. Read `lunar/apollo_helpers.py` to see how raw data becomes T_{eq} .
13. Skim the paper abstract, plain-language summary, and §2 (methods) in `paper/letter/letter.tex`.

Then — make a change (the real test of understanding)

14. Try a tiny experiment: in a scratch script, call `run_with(SITES['A15'], kd=...)` at a couple of K_d values and print the temperature at 1.4 m. Watch it move toward the data as K_d approaches 4.58.
15. Read `docs/FLAG_REPORT.md` to see how the project audits *itself* — this is how careful computational science is actually done.

Deeper study, when ready

- **Numerical methods:** the finite-volume method, implicit vs explicit schemes, stability (any intro numerical-PDE text).
- **Heat transfer:** Fourier’s law and the 1-D heat equation (any intro heat-transfer text).
- **Python scientific stack:** NumPy basics (arrays, broadcasting, `np.interp`, `np.gradient`) — used everywhere here.
- **Statistics:** the bootstrap and confidence intervals.

The single most effective habit: keep this guide open on one side and the actual code file on the other. Every concept here points at a real function you can open, run, and modify. Understanding grows fastest when the words and the code are side by side.